FINAL Unit2 MCQ - Topic order

1. Users of a mobile application are categorized based on the average number of hours they use the application each day, as shown in the following table.

Category	Average Hours of Use Per Day
Top User	Greater than 5
Regular User	Between 1 and 5, inclusive
Inactive User	Less than 1

Suppose *n* is the average number of hours a user spends using the application each day. Which of the following algorithms can be used to assign the correct categorization to a user?

(A) If *n* is greater than 5, categorize the user as a top user. Otherwise, if *n* is less than or equal to 5, categorize the user as a regular user. Otherwise, categorize the user as an inactive user.

(B) If *n* is greater than 5, categorize the user as a top user. Otherwise, if *n* is greater than or equal to 1, categorize the user as a regular user. Otherwise, categorize the user as an inactive user. ✓

(C) If *n* is less than 1, categorize the user as an inactive user. Otherwise, if *n* is greater than or equal to 1, categorize the user as a regular user. Otherwise, categorize the user as a top user.

(D) If *n* is greater than or equal to 1, categorize the user as a regular user. Otherwise, if *n* is less than 1, categorize the user as an inactive user. Otherwise, categorize the user as a top user.

Answer B

Correct. This algorithm first checks if *n* is greater than 5, and if so, correctly categorizes the user as a top user. If this is not the case, the algorithm then checks if *n* is between 1 and 5, inclusive, and if so, correctly categorizes the user as a regular user. If this is not the case, the algorithm then correctly categorizes the user as an inactive user.

FINAL Unit2 MCQ - Topic order

2. Which of the following algorithms can be used to display exactly 10 random even numbers between 0 and 100, inclusive?

(A) Repeat the following 100 times: generate a random number between 0 and 10, inclusive. If the number is even, display it.

(B) Repeat the following 10 times: generate a random number between 0 and 100, inclusive, multiply it by 2, and display the result.

(C) Repeat the following 10 times: generate a random number between 0 and 100, inclusive. If the number is even, display it.

(D) Repeat the following 10 times: generate a random number between 0 and 50, multiply it by 2, and display the result. ✓

Answer D

Correct. Generating numbers between 0 and 50 and then multiplying by 2 produces even numbers between 0 and 100. Repeating this 10 times ensures that exactly 10 even numbers are displayed.

3. A program is being written to control access to a secure facility. The gate to the facility should open only when the user has a valid access badge and the current hour is between 7 and 18, inclusive.

Which of the following algorithms can be used to determine whether to allow access to the facility?

(A) If the person has a valid access badge and the hour is at least 7 and at most 18, then open the gate; otherwise, deny access. ✓

(B) If the person has a valid access badge and the hour is greater than 7 and less than 18, then open the gate; otherwise, deny access.

(C) If the person does not have a valid access badge and the hour is at least 7 and at most 18, then deny access; otherwise, open the gate.

(D) If the person does not have a valid access badge and the hour is less than 7 and greater than 18, then deny access; otherwise, open the gate.

Answer A

Correct. This algorithm matches the requirement in that the person must have a valid access badge and the hour must be between 7 and 18, inclusive.

FINAL Unit2 MCQ - Topic order

4. A company is designing a program to evaluate the strength of a user's password. The program needs to indicate if the password is "weak," "medium," or "strong" based on the following guidelines.

- A password with fewer than 8 characters is classified as "weak."
- A password with 8 or more characters is classified as "medium," where all characters are either numbers or letters.
- A password with 8 or more characters plus at least one special character (like !, @, or #) is classified as "strong."

Which of the following algorithms will correctly classify a password?

(A) Step 1: If the password has fewer than 8 characters, label it as "weak" and skip Steps 2 and 3. Otherwise, go to Step 2.

Step 2: If the password has more than 8 characters, label it as "medium" and skip Step 3. Otherwise, go to Step 3.

Step 3: Label the password as "strong."

(B) ✓ Step 1: If the password has fewer than 8 characters, label it as "weak" and skip Steps 2 and 3. Otherwise, go to Step 2.

Step 2: If the password contains at least one special character, label it as "strong" and skip Step 3. Otherwise, go to Step 3.

Step 3: Label the password as "medium."

(C) Step 1: If the password has 8 or more characters, label it as "medium" and skip Steps 2 and 3. Otherwise, go to Step 2.

Step 2: If the password has more than 8 characters and contains at least one special character, label it as "strong" and skip Step 3. Otherwise, go to Step 3.

Step 3: Label the password as "weak."

(D) Step 1: If the password has 8 or more characters and contains at least one special character, label it as "strong" and skip Steps 2 and 3. Otherwise, go to Step 2.

Step 2: If the password does not contain at least one special character, label it as "medium" and skip Step 3. Otherwise, go to Step 3.

Step 3: Label the password as "weak."

Answer B

Correct. This algorithm will yield correct results each time. The algorithm first labels all passwords shorter than 8 characters as "weak" and then skips the other steps. If a password is not labeled weak, the algorithm then considers if the password has a special character and labels those "strong", and then skips the last step. Finally, if a password was not labeled "weak" or "strong", the algorithm labels the password "medium."

FINAL Unit2 MCQ - Topic order

5. A programmer wants to create a fitness application that counts the number of days in a month that a user walks at least 10,000 steps. Which of the following algorithms can be used to determine the correct count?

(A) At the beginning of the month, start with a day count of 0 and a total step count of 0. At the end of each day, add 1 to the day count, check the number of steps taken that day, and add it to the total step count. At the end of the month, if the total step count is at least 10,000, output the day count.

(B) At the beginning of the month, start with a day count of 0. At the end of each day, check the number of steps taken. Set the day count equal to the number of steps taken that day. At the end of the month, if the day count is at least 10,000, output the day count.

(C) At the beginning of the month, start with a day count of 0. At the end of each day, check the number of steps taken. If the number of steps taken that day is at least 10,000, set the day count equal to the number of steps taken that day. At the end of the month, output the day count.

(D) At the beginning of the month, start with a day count of 0. At the end of each day, check the number of steps taken. If the number of steps taken that day is at least 10,000, add 1 to the day count. At the end of the month, output the day count. ✓

Answer D

Correct. This algorithm determines the number of steps taken each day and counts only the days where the daily step count is over 10,000.

FINAL Unit2 MCQ - Topic order

6. The variables x, y, and z are integers. Which of the following algorithms can be used to swap the values of the variables so that they are ordered from least to greatest (i.e., $x \leq y \leq z$)?

FINAL Unit2 MCQ - Topic order

(A)

Step 1:	If x is greater than y, swap the values of x and y.
Step 2:	If y is greater than z, swap the values of y and z.

(B)

Step 1:	If x is greater than z, swap the values of x and z.
Step 2:	If y is greater than z, swap the values of y and z.

(C) ✓

Step 1:	If x is greater than y, swap the values of x and y.
Step 2:	If y is greater than z, swap the values of y and z.
Step 3:	If x is greater than y, swap the values of x and y.

FINAL Unit2 MCQ - Topic order

(D)

Step 1:	If x is greater than y, swap the values of x and y.
Step 2:	If y is greater than z, swap the values of y and z.
Step 3:	If x is greater than z, swap the values of x and z.

Answer C

Correct. Step 1 ensures that x is less than or equal to y. Step 2 ensures that z is the largest number. Step 3 ensures that x is the smallest number. After all three steps are executed, the variables will be in the desired order.

7. Consider the following declarations.

```
int valueOne, valueTwo;
```

Assume that `valueOne` and `valueTwo` have been initialized. Which of the following evaluates to `true` if `valueOne` and `valueTwo` contain the same value?

(A) `valueOne.equals((Object) valueTwo)`

(B) `valueOne == valueTwo` ✓

(C) `valueOne.compareTo(valueTwo) == 0`

(D) `valueOne.equals(valueTwo)`

FINAL Unit2 MCQ - Topic order

8. In the following code segment, a and b are properly declared and initialized boolean variables.

```
boolean result = (a != b);
System.out.println(result);
```

Which of the following best describes the behavior of the code segment?

(A) It prints true if a is true and b is false, and prints false otherwise.

(B) It prints true if a and b have different values, and prints false otherwise. ✓

(C) It prints true if a and b have the same value, and prints false otherwise.

(D) It prints false for all possible values of a and b.

Answer B

Correct. The expression a != b evaluates to true when a and b are not equal. It evaluates to false when a and b are equal.

FINAL Unit2 MCQ - Topic order

9. Consider the following code segment.

```
int a = 10;
int b = 5 * 2;
System.out.print(a == b);
```

What is printed as a result of executing the code segment?

(A)

10

(B)

10 == 10

(C) ✓

true

(D)

false

Answer C

Correct. After the second line of the code segment executes, `a` has the value `10` and `b` also has the value `10`, so `a == b` evaluates to `true`.

FINAL Unit2 MCQ - Topic order

10. Consider the following code segment, which is intended to print `true` only when both `r` and `v` are positive numbers. Assume that `r` and `v` have been properly declared and initialized.

```
boolean rPos = r > 0;
boolean vPos = v > 0;
System.out.print(rPos == vPos);
```

The code segment does not always work as intended.

Which of the following values for `r` and `v` will demonstrate that this code segment does **not** work as intended?

(A)

r = -10, v = 10

(B) ✓

r = -1, v = -4

(C)

r = 5, v = -2

(D)

r = 6, v = 3

Answer B

Correct. For these values, the code is intended to print `false` since variables are negative. `rPos` will evaluate to `false`, and `vPos` will evaluate to `false`. The expression `rPos == vPos` will evaluate to `true`. This is not the intention of the code.

FINAL Unit2 MCQ - Topic order

11. Consider the following statement.

boolean x = (5 < 8) == (5 == 8);

What is the value of x after the statement is executed?

(A)

5

(B)

8

(C)

true

(D) ✓

false

Answer D

Correct. Since the value of the boolean expression (5 < 8) is true and the value of the boolean expression (5 == 8) is false, the value of the expression (5 < 8) == (5 == 8) is equivalent to the value of the expression true == false, which is false.

FINAL Unit2 MCQ - Topic order

12. In the following Boolean expression, the int variables x and y have been properly declared and initialized.

(x <= 10) == (y > 25)

Which of the following values for x and y will result in the expression evaluating to true?

(A) x = 8 and y = 25

(B) x = 10 and y = 10

(C) x = 10 and y = 30 ✓

(D) x = 15 and y = 30

Answer C

Correct. When x is 10, the expression x <= 10 evaluates to true. When y is 30, the expression y > 25 evaluates to true. Therefore, the expression (x <= 10) == (y > 25) compares true == true, resulting in true. The expression will evaluate to true only when both sides of the == operator have the same value (either both true or both false).

FINAL Unit2 MCQ - Topic order

13. Consider the following incomplete method, which is intended to return `true` if the value of `y` is between the values of the other two parameters and `false` otherwise.

```
/** Precondition: x, y, and z have 3 different values. */
public static boolean compareThree(int x, int y, int z)
{
 return /* missing condition */ ;
}
```

The following table shows the results of several calls to `compareThree`.

Call	Result
`compareThree(4, 5, 6)`	`true`
`compareThree(6, 5, 4)`	`true`
`compareThree(5, 4, 6)`	`false`
`compareThree(3, 4, 4)`	violates precondition

Which of the following can be used to replace `/* missing condition */` so that `compareThree` will work as intended when called with parameters that satisfy its precondition?

(A) `(x > y) && (y > z)`

(B) `(x > y) || (y > z)`

(C) `(x > y) == (y > z)` ✓

(D) `(x > y) != (y > z)`

Answer C

Correct. The value of `y` is between the values of `x` and `z` when the three parameters are in either increasing or decreasing order. When the three parameters are in increasing order, the expressions `x > y` and `y > z` are both `false`. When the three parameters are in decreasing order, the expressions `x > y` and `y > z` are both `true`. The expression `(x > y) == (y > z)` evaluates to `true` when both operands of the `==` operator are the same.

FINAL Unit2 MCQ - Topic order

14. In the following statement, j and k are properly declared and initialized int variables.

boolean result = ((j > 0) && (k > 0)) || ((j < 0) && (k < 0));

Which of the following best describes the conditions in which result is assigned the value true?

(A) When j and k are both equal to 0

(B) When j and k are both positive or both negative ✓

(C) When j is greater than k

(D) When j is positive and k is negative or when j is negative and k is positive

Answer B

Correct. The part of the Boolean expression before the || evaluates to true when both j and k are positive. The part of the Boolean expression after the || evaluates to true when both j and k are negative. Therefore, the whole expression evaluates to true when j and k are both positive or both negative.

15. Consider the following code segment.

```
int x = /* some integer value */ ;
int y = /* some integer value */ ;

boolean result = (x < y);

result = ( (x >= y) && !result );
```

Which of the following best describes the conditions under which the value of `result` will be `true` after the code segment is executed?

(A) Only when `x >= y` ✓

(B) Only when `x` and `y` are equal

(C) The value will always be `true`.

(D) The value will never be `true`.

FINAL Unit2 MCQ - Topic order

16. Consider the following method.

```
public int someCode(int a, int b, int c)
{
  if ((a < b) && (b < c))
    return a;
  if ((a >= b) && (b >= c))
    return b;
  if ((a == b) || (a == c) || (b == c))
    return c;
}
```

Which of the following best describes why this method does not compile?

(A) It is possible to reach the end of the method without returning a value. ✓

(B) The `if` statements must have `else` parts when they contain `return` statements.

(C) Methods cannot have multiple `return` statements.

(D) The third `if` statement is not reachable.

FINAL Unit2 MCQ - Topic order

17. Consider the following code segment, in which `m` and `n` are properly declared and initialized `int` variables.

```
int result = 0;
for (int i = 0; i < n; i++)
{
  for (int j = m; j < n; j++)
  {
    result++;
  }
}
```

Which of the following best describes the conditions that must be true of `m` and `n` so that after this code segment executes, the value of `result` is greater than 0?

(A) `n > 0` and `m > 0`

(B) `n > 0` and `m > n`

(C) `n > 0` and `n > m` ✓

(D) `m > 0` and `m > n`

Answer C

Correct. If `n` is not greater than 0, the outer loop will never execute. If `n` is not greater than `m`, the inner loop will never execute.

FINAL Unit2 MCQ - Topic order

18. The following code segment is intended to print the total cost of a stay in a room at a particular hotel. The int variable numNights represents the length of the stay, in nights, and the double variable dailyRate represents the room base cost for each night of the stay. For stays that are longer than five nights, there is a 10% discount applied to the room base cost for all nights during the stay. In addition to the room cost, the hotel charges a $25 resort fee for each night of the stay. The resort fee is not eligible for the 10% discount for stays that are longer than five nights.

```
int numNights =    /* some initial value */ ;
double dailyRate = /* some initial value */ ;

double totalCost = numNights * dailyRate;
/* missing code */

System.out.println(totalCost);
```

Which of the following can be used to replace /* missing code */ so that the code segment works as intended?

FINAL Unit2 MCQ - Topic order

(A)

```
if (numNights > 5)
{
  totalCost *= 0.9;
  totalCost += (numNights * 25);
}
```

(B) ✓

```
if (numNights > 5)
{
  totalCost *= 0.9;
}
totalCost += (numNights * 25);
```

(C)

```
totalCost += (numNights * 25);
if (numNights > 5)
{
  totalCost *= 0.9;
}
```

FINAL Unit2 MCQ - Topic order

(D)

```
totalCost += 25;

if (numNights > 5)

{

  totalCost *= 0.9;

}
```

Answer B

Correct. This code segment first applies the 10% discount (by multiplying by 0.9) if the stay is longer than 5 nights, and then adds the resort fee ($25 per night) for all nights.

FINAL Unit2 MCQ - Topic order

19. Consider the following code segment, which is intended to simulate a random process. The code is intended to set the value of the variable `event` to exactly one of the values `1`, `2`, or `3`, depending on the probability of an event occurring. The value of `event` should be set to `1` if the probability is 70 percent or less. The value of `event` should be set to `2` if the probability is greater than 70 percent but no more than 80 percent. The value of `event` should be set to `3` if the probability is greater than 80 percent. The variable `randomNumber` is used to simulate the probability of the event occurring.

```
int event = 0;
if (randomNumber <= 0.70)
{
 event = 1;
}
if (randomNumber <= 0.80)
{
 event = 2;
}
else
{
 event = 3;
}
```

The code does not work as intended. Assume that the variable `randomNumber` has been properly declared and initialized. Which of the following initializations for `randomNumber` will demonstrate that the code segment will not work as intended?

(A) `randomNumber = 0.70;` ✓

(B) `randomNumber = 0.80;`

(C) `randomNumber = 0.85;`

(D) `randomNumber = 1.00;`

Answer A

Correct. When `randomNumber` is `0.70`, the value of `event` should be set to `1`, but the code segment does not work as intended. In the first `if` statement, the expression `randomNumber <= 0.7` evaluates to `true`, and `event` is correctly set to `1`. In the `if-else` statement, the expression `randomNumber <= 0.8` evaluates to `true`, and `event` is incorrectly set to `2`. Therefore, this shows that the code does not work as intended.

FINAL Unit2 MCQ - Topic order

20. Consider the following code segment.

```
int temp = 75;
String choice;
boolean hasSibling = false;

if (temp < 70)
{
  choice = "Indoors";
}
else
{
  choice = "Outdoors";
}
if (hasSibling)
{
  choice += " with sibling";
}
```

What is the value of `choice` after executing the code segment?

(A)

"Indoors"

(B) ✓

"Outdoors"

(C)

"Indoors with sibling"

(D)

"Outdoors with sibling"

Answer B

Correct. Since `75 < 70` is not `true`, the first `if` statement evaluates to `false`, which results in the variable `choice` being assigned the value `"Outdoors"`. The second `if` statement results in `false` since `hasSibling` is assigned the value `false`, so no additional assignment statement is executed.

FINAL Unit2 MCQ - Topic order

21. Consider the following code segment.

```
int x = 3;
int y = -1;

if (x - 2 > y)
{
  x -= y;
}
if (y + 3 >= x)
{
  y += x;
}

System.out.print("x = " + x + ", y = " + y);
```

What is printed as a result of executing the code segment?

(A)

x = 2, y = 1

(B)

x = 3, y = 2

(C) ✓

x = 4, y = -1

(D)

x = 4, y = 3

Answer C

Correct. The Boolean expression in the first `if` statement evaluates to `true`, so the statement `x -= y` is executed, setting `x` equal to `x - y`, or `4`. The Boolean expression in the second `if` statement evaluates to `false`. Therefore, the statement `y += x` is not executed. The values of `x` and `y` are `4` and `-1`, respectively, when the `print` statement is executed.

FINAL Unit2 MCQ - Topic order

22. Consider the following two code segments where the int variable choice has been properly declared and initialized.

Code Segment 1

```
if (choice > 10)
{
  System.out.println("blue");
}
else if (choice < 5)
{
  System.out.println("red");
}
else
{
  System.out.println("yellow");
}
```

Code Segment 2

```
if (choice > 10)
{
  System.out.println("blue");
}
if (choice < 5)
{
  System.out.println("red");
}
else
{
  System.out.println("yellow");
}
```

Assume that both code segments initialize choice to the same integer value. Which of the following best describes the conditions on the initial value of the variable choice that will cause the two code segments to produce different output?

(A) choice > 5 and choice < 10

(B) choice > 10 ✓

(C) choice == 5 or choice == 10

(D) There is no value for choice that will cause the two code segments to produce different output.

Answer B

FINAL Unit2 MCQ - Topic order

Correct. When choice is greater than 10, code segment 1 will print "blue" and the else statements are not executed. Code segment 2 will print "blue" but will then execute the next if statement and print "yellow", giving different output for initial values that are greater than 10.

23. Consider the following method.

```
public String exercise(int input)
{
 if (input < 10)
 {
   return "alpha";
 }
 if (input < 5)
 {
   return "beta";
 }
 if (input < 1)
 {
   return "gamma";
 }
 return "delta";
}
```

Assume that the `int` variable `x` has been initialized in another method in the same class. Which of the following describes the conditions under which the method call `exercise(x)` will return `"gamma"` ?

(A) The method will never return `"gamma"`. ✓

(B) The method will return `"gamma"` if `x` is less than `1`.

(C) The method will return `"gamma"` if `x` is between `1` and `4`, inclusive.

(D) The method will always return `"gamma"`.

Answer A

Correct. If `x` is less than `10`, then the body of the first `if` statement is executed and the method returns `"alpha"`. If `x` is greater than or equal to `10`, neither of the Boolean conditions in the next two `if` statements evaluates to `true`, so `"delta"` is returned.

FINAL Unit2 MCQ - Topic order

24. The following method is intended to return `true` if and only if the parameter `val` is a multiple of `4` but is not a multiple of `100` unless it is also a multiple of `400`. The method does not always work correctly.

```
public boolean isLeapYear(int val)
{
 if ((val % 4) == 0)
 {
   return true;
 }
 else
 {
   return (val % 400) == 0;
 }
}
```

Which of the following method calls will return an incorrect response?

(A) `isLeapYear(1900)` ✓

(B) `isLeapYear(1984)`

(C) `isLeapYear(2000)`

(D) `isLeapYear(2001)`

Answer A

Correct. The value `1900` is a multiple of `4`, so the expression `(val % 4) == 0` evaluates to `true` and the statement `return true;` is executed, thereby exiting the method with a return value of `true`. This is an error because even though `1900` is a multiple of `100`, it is not a multiple of `400`, so the method should have returned `false`. Any value that is a multiple of `100` but not a multiple of `400` will pass the first test and return `true` even though the method was intended to return `false`.

FINAL Unit2 MCQ - Topic order

25. In the code segment below, the `int` variable `temp` represents a temperature in degrees Fahrenheit. The code segment is intended to print a string based on the value of `temp`. The following table shows the string that should be printed for different temperature ranges.

Temperature Range	String to Print
31 and below	`"cold"`
32–50	`"cool"`
51–70	`"moderate"`
71 and above	`"warm"`

```
String weather;
if (temp <= 31)
{
 weather = "cold";
}
else
{
 weather = "cool";
}
if (temp >= 51)
{
 weather = "moderate";
}
else
{
 weather = "warm";
}
System.out.print(weather);
```

Which of the following test cases can be used to show that the code does NOT work as intended?

I. `temp = 30`
II. `temp = 51`
III. `temp = 60`

(A) I only ✓

(B) I and II only

(C) II and III only

(D) I, II, and III

Answer A

FINAL Unit2 MCQ - Topic order

Correct. If `temp` is `30`, the `else`-clause of the second conditional statement assigns the value `"warm"` to `weather`, since the condition `temp >= 51` evaluates to `false`. However, the intended value is `"cold"`. If `temp` is `51` or `60`, the `if`-clause of the second conditional statement assigns the value `"moderate"` to `weather`, since the condition `temp >= 51` evaluates to `true`. This is the intended result.

26. A school that does not have air conditioning has published a policy to close school when the outside temperature reaches or exceeds 95°F. The following code segment is intended to print a message indicating whether or not the school is open, based on the temperature. Assume that the variable `degrees` has been properly declared and initialized with the outside temperature.

```
if (degrees > 95)
{
 System.out.println("School will be closed due to extreme heat");
}
else
{
 System.out.println("School is open");
}
```

Which of the following initializations for `degrees`, if any, will demonstrate that the code segment may not work as intended?

(A) `degrees = 94;`

(B) `degrees = 95;` ✓

(C) `degrees = 96;`

(D) The code will work as intended for all values of `degrees`.

Answer B

Correct. When `degrees` is `95`, the expression `degrees > 95` evaluates to `false` so that `"School is open"` is printed. This shows that the code does not work as intended, because it was supposed to show that school was closed when the temperature reached or exceeded 95°F.

FINAL Unit2 MCQ - Topic order

27. Consider the following code segment.

```
int x = 7;
if (x < 7)
{
  x = 2 * x;
}
if (x % 3 == 1)
{
  x = x + 2;
}
System.out.print(3 * x);
```

What is printed as a result of executing the code segment?

(A) `9`

(B) `14`

(C) `21`

(D) `27` ✓

Answer D

Correct. The initial value of `x` is `7`. The condition `(x < 7)` is `false`, so the body of the first `if` statement is not executed. The condition `x % 3 == 1` is `true`, so the body of the second `if` statement is executed. This increases the value of `x` from `7` to `9`. Lastly, the value of `3 * x`, which is `27`, is printed.

FINAL Unit2 MCQ - Topic order

28. In the following code segment, n is a properly declared and initialized int variable.

```
boolean result = false;
if (n >= 10)
{
  result = true;
}
if (n <= 50)
{
  result = true;
}
System.out.println(result);
```

Which of the following best describes the behavior of the code segment?

(A) It prints true for all possible values of n. ✓

(B) It prints false for all possible values of n.

(C) It prints true if n is between 10 and 50, inclusive, and prints false otherwise.

(D) It prints false if n is between 10 and 50, inclusive, and prints true otherwise.

Answer A

Correct. The code segment initializes result to false, then uses the two if statements to set result to true if n is greater than or equal to 10 or less than or equal to 50. All possible values of n will make at least one of these conditions true, so the code segment will always print true.

FINAL Unit2 MCQ - Topic order

29. The following method is intended to return "Renaissance" for years from 1400 to 1600, inclusive, "Medieval" for years from 400 to 1399, inclusive, or "Other" for any other year. In the method, the parameter represents a year.

```
public static String getCategory(int year)
{
  String category = "";
  /* missing code */
  return category;
}
```

Which of the following code segments can replace /* missing code */ so that the getCategory method works as intended?

FINAL Unit2 MCQ - Topic order

(A)

```
if (year > 1600 || year < 400)
{
  category = "Other";
}
else if (year >= 1400)
{
  category = "Renaissance";
}
else if (year >= 400)
{
  category = "Medieval";
}
```

FINAL Unit2 MCQ - Topic order

(B)

```
if (year > 1600 || year < 400)
{
  category = "Other";
}
if (year >= 1400)
{
  category = "Renaissance";
}
if (year >= 400)
{
  category = "Medieval";
}
```

FINAL Unit2 MCQ - Topic order

(C)

```
if (year >= 1400)
{
  category = "Renaissance";
}
else if (year >= 400)
{
  category = "Medieval";
}
else
{
  category = "Other";
}
```

FINAL Unit2 MCQ - Topic order

(D)

```
if (year >= 1400)
{
  category = "Renaissance";
}
if (year >= 400)
{
  category = "Medieval";
}
else
{
  category = "Other";
}
```

Answer A

Correct. In this code segment, the variable category is assigned "Other" if year is greater than 1600 or less than 400. If not, then if year is greater than or equal to 1400, category is assigned "Renaissance". If not, then if year is greater than or equal to 400, category is assigned "Medieval".

FINAL Unit2 MCQ - Topic order

30. A computer game should print different messages based on a player's score.

- "Great!" is printed for scores greater than 500.
- "Good!" is printed for scores between 250 and 500, inclusive.
- "Try again." is printed for scores less than 250.

Assume that score is a properly declared and initialized int that represents a player's score.
Which of the following code segments will print the correct message for any valid value of score?

FINAL Unit2 MCQ - Topic order

(A) ✓

```
String message = "";

if (score < 250)

{

  message = "Try again.";

}

else if (score <= 500)

{

  message = "Good!";

}

else

{

  message = "Great!";

}

System.out.println(message);
```

FINAL Unit2 MCQ - Topic order

(B)

```
String message = "";

if (score < 250)

{

  message = "Try again.";

}



if (score <= 500)

{

  message = "Good!";

}

else

{

  message = "Great!";

}

System.out.println(message);
```

FINAL Unit2 MCQ - Topic order

(C)

```
String message = "";

if (score >= 250)

{

  message = "Good!";

}

else if (score > 500)

{

  message = "Great!";

}

else

{

  message = "Try again.";

}

System.out.println(message);
```

FINAL Unit2 MCQ - Topic order

(D)

```
String message = "";

if (score > 500)

{

  message = "Great!";

}

else if (score <= 500)

{

  message = "Good!";

}

else

{

  message = "Try again.";

}

System.out.println(message);
```

Answer A

Correct. In this code segment, the if clause assigns "Try again." to message for scores less than 250. The else if clause assigns "Good!" to message for scores that are less than or equal to 500 but not handled by the previous if clause. Therefore, message is assigned "Good!" for scores greater than or equal to 250 and less than or equal to 500. The else clause assigns "Great!" to message for scores not handled by the previous if and else if clauses. Therefore, message is assigned "Great!" for scores greater than 500.

FINAL Unit2 MCQ - Topic order

31. Consider the following two code segments. Assume that variables x and y have been declared as int variables and have been assigned integer values.

Code Segment I

```
int result = 0;
if (x > y)
{
  result = x - y;
  System.out.print(result);
}
else if (x < y)
{
  result = y - x;
  System.out.print(result);
}
else
{
  System.out.print(result);
}
```

Code Segment II

```
if (x < y)
{
  System.out.print(y - x);
}
else
{
  System.out.print(x - y);
}
```

Which of the following correctly compares the outputs of the two code segments?

(A) Code segment I and code segment II produce the same output for all values of x and y. ✓

(B) Code segment I and code segment II produce the same output only when x is equal to y.

(C) Code segment I and code segment II produce the same output only when x is not equal to y.

(D) Code segment I and code segment II produce the same output only when x is less than y.

Answer A

Correct. Both code segments print the absolute value of the difference between x and y. When x is greater than y, both code segments print x - y. When x is less than y, both code segments print y - x. When x is equal

to y, both segments print 0.

32. Consider the following method, which returns an `int` based on its parameter `x`.

```
public static int puzzle(int x)
{
  if (x > 20)
  {
    x -= 2;
  }
  else if (x % 2 == 0) // Line 7
  {
    x += 4;
  }
  return x;
}
```

Consider a modification to the method that eliminates the `else` from line 7 so that line 7 becomes

```
if (x % 2 == 0) // Modified line 7
```

For which of the following values of `x` would the return values of the original method and the modified method differ?

(A) `5`

(B) `14`

(C) `22` ✓

(D) `25`

Answer C

Correct. In the original method, the call `puzzle(22)` returns `20`. The first condition evaluates to `true`, so `x` is decreased by two and `20` is returned. In the original method, when the condition in the `if` statement evaluates to `true`, the second condition, in the `else if` clause, is not evaluated. For the modified method, the first condition still evaluates to `true` and `x` is decreased by two. But now the second condition appears in an `if` statement, instead of in an `else if` clause, so the second condition is evaluated, found to be `true`, and `x` is increased by four. This results in the value `24` being returned.

FINAL Unit2 MCQ - Topic order

33. In the following code segment, the `int` variables `a`, `b`, `c`, and `d` have been properly declared and initialized.

```
if (a < b || c != d)
{
  System.out.println("dog");
}
else
{
  System.out.println("cat");
}
```

Which of the following code segments produces the same output as the given code segment for all values of `a`, `b`, `c`, and `d`?

FINAL Unit2 MCQ - Topic order

(A)

```
if (a < b && c != d)
{
  System.out.println("dog");
}
else
{
  System.out.println("cat");
}
```

(B)

```
if (a > b && c == d)
{
  System.out.println("cat");
}
else
{
  System.out.println("dog");
}
```

FINAL Unit2 MCQ - Topic order

(C)

```
if (a >= b || c == d)
{
  System.out.println("cat");
}
else
{
  System.out.println("dog");
}
```

(D) ✓

```
if (a >= b && c == d)
{
  System.out.println("cat");
}
else
{
  System.out.println("dog");
}
```

Answer D

Correct. Since the original expression prints `"dog"` when `a < b || c != d` evaluates to `true`, an equivalent code segment would print `"cat"` when the negative of the expression, or `!(a < b || c != d)`, evaluates to `true`. Applying De Morgan's law produces the equivalent Boolean expression `a >= b && c == d` for the case when `"cat"` should be printed.

FINAL Unit2 MCQ - Topic order

34. Consider the following code segment.

```
int k = 0;
while (k < 10)
{
  System.out.print((k % 3) + " ");
  if ((k % 3) == 0)
    k = k + 2;
  else
    k++;
}
```

What is printed as a result of executing the code segment?

(A) `0 2 0 2 0 2`

(B) `0 2 1 0 2 1 0`

(C) `0 2 0 2 0 2 0` ✓

(D) `0 1 2 1 2 1 2`

FINAL Unit2 MCQ - Topic order

35. Consider the following code segments, which are each intended to convert grades from a 100-point scale to a 4.0-point scale and print the result. A grade of 90 or above should yield a 4.0, a grade of 80 to 89 should yield a 3.0, a grade of 70 to 79 should yield a 2.0, and any grade lower than 70 should yield a 0.0.

Assume that `grade` is an `int` variable that has been properly declared and initialized.

Code Segment 1

```
double points = 0.0;
if (grade > 89)
{
  points = 4.0;
}
else if (grade > 79)
{
  points = 3.0;
}
else if (grade > 69)
{
  points = 2.0;
}
else
{
  points = 0.0;
}
System.out.println(points);
```

Code Segment 2

```
double points = 0.0;
if (grade > 89)
{
  points = 4.0;
}
if (grade > 79)
{
  points = 3.0;
}
if (grade > 69)
{
  points = 2.0;
}
if (grade < 70)
{
  points = 0.0;
```

FINAL Unit2 MCQ - Topic order

```
}
System.out.println(points);
```

Which of the following statements correctly compares the values printed by the two code segments?

(A) The two code segments print the same value only when `grade` is below `80`. ✓

(B) The two code segments print the same value only when `grade` is `90` or above.

(C) Both code segments print the same value for all possible values of `grade`.

(D) The two code segments print different values for all possible values of `grade`.

Answer A

Correct. Because code segment 1 uses `else` statements, `points` is assigned a new value at most one time after it is initialized. Because code segment 2 does not use `else` statements, it is possible for multiple conditions to evaluate to `true`, causing `points` to be assigned a new value multiple times. For grades lower than `70`, both code segments print `0.0`. For grades from `70` to `79`, both code segments print `2.0`. For grades from `80` to `89`, code segment 1 prints `3.0` and code segment 2 prints `2.0`. For grades `90` or above, code segment 1 prints `4.0` and code segment 2 prints `2.0`.

FINAL Unit2 MCQ - Topic order

36. The following categories are used by some researchers to categorize regions as urban, suburban, or rural based on population density.

- An urban region is a region with more than 3,000 people per square mile.
- A suburban region is a region with between 1,000 and 3,000 people, inclusive, per square mile.
- A rural region is a region with fewer than 1,000 people per square mile.

Consider the following method, which is intended to categorize a region as urban, suburban, or rural based on the population density of the region.

```
public static String getCategory(int density)
{
  /* missing code */
}
```

Which of the following code segments can replace /* missing code */ so the getCategory method works as intended?

FINAL Unit2 MCQ - Topic order

(A) ✓

```
String category;

if (density > 3000)

{

  category = "urban";

}

else if (density >= 1000)

{

  category = "suburban";

}

else

{

  category = "rural";

}

return category;
```

FINAL Unit2 MCQ - Topic order

(B)

```
String category;

if (density > 3000)

{

  category = "urban";

}

else if (density <= 3000)

{

  category = "suburban";

}

else

{

  category = "rural";

}

return category;
```

FINAL Unit2 MCQ - Topic order

(C)

```
String category = "rural";

if (density > 3000)

{

  category = "urban";

}

if (density >= 1000)

{

  category = "suburban";

}

return category;
```

(D)

```
String category = "urban";

if (density < 1000)

{

  category = "rural";

}

if (density >= 1000)

{

  category = "suburban";

}

return category;
```

FINAL Unit2 MCQ - Topic order

Answer A

Correct. This code segment assigns "urban" to category when density is greater than 3000. Otherwise, it assigns "suburban" to category when density is greater than or equal to 1000 (i.e., when density is between 1000 and 3000, inclusive). Otherwise, it assigns "rural" to category (i.e., when density is less than 1000).

FINAL Unit2 MCQ - Topic order

37. In the following code segment, str is a properly declared and initialized String variable.

```
int result = 0;
if (str.length() > 5)
{
  if (str.indexOf("A") < 0)
  {
    result = 1;
  }
  else if (str.indexOf("B") < 0)
  {
    result = 2;
  }
}
else if (str.indexOf("A") < 0)
{
  result = 3;
}
```

Which of the following code segments assigns the same value to result as the preceding code segment for all values of str?

FINAL Unit2 MCQ - Topic order

(A)

```
int result = 0;

if (str.indexOf("A") < 0)

{

  result = 3;

}

else if (str.indexOf("B") < 0)

{

  result = 2;

}

else if (str.length() > 5);

{

  result = 1;

}
```

FINAL Unit2 MCQ - Topic order

(B)

```
int result = 0;
if (str.indexOf("A") < 0)
{
  if (str.length() > 5)
  {
    result = 1;
  }
  else if (str.indexOf("B") < 0)
  {
    result = 2;
  }
  else
  {
    result = 3;
  }
}
```

FINAL Unit2 MCQ - Topic order

(C) ✓

```
int result = 0;
if (str.indexOf("A") < 0)
{
  if(str.length() > 5)
  {
    result = 1;
  }
  else
  {
    result = 3;
  }
}
else if (str.indexOf("B") < 0)
{
  if (str.length() > 5)
  {
    result = 2;
  }
}
```

FINAL Unit2 MCQ - Topic order

(D)

```
int result = 0;
if (str.indexOf("B") < 0)
{
  if (str.length() > 5)
  {
    result = 2;
  }
}
else if (str.indexOf("A") < 0)
{
  if (str.length() > 5)
  {
    result = 1;
  }
  else
  {
    result = 3;
  }
}
```

Answer C

Correct. The code will assign result to the correct value for all possible string values.

FINAL Unit2 MCQ - Topic order

38. Consider the following code segment.

```
double cost = 25.0;
double discount = 0.0;

if (cost >= 100.0)
{
  discount = 10.0;
}
else if (cost >= 50.0)
{
  discount = 5.0;
}
else
{
  discount = 2.0;
}

System.out.println(cost - discount);
```

What is printed as a result of executing this code segment?

(A)

25.0

(B) ✓

23.0

(C)

20.0

(D)

15.0

Answer B

Correct. Since cost is 25.0, the expressions cost >= 100 and cost >= 50 both evaluate to false. As a result, the final else clause is executed, assigning 2.0 to discount. The print statement prints the value of the expression 25.0 - 2.0, which is 23.0.

FINAL Unit2 MCQ - Topic order

39. Consider the following code segment.

```
int start = 4;
int end = 5;

if (start < end)
{
  if (end > 0)
  {
    start += 2;
    end++;
  }
  else
  {
    end += 3;
  }
}

if (start < end)
{
  if (end == 0)
  {
    end += 2;
    start++;
  }
  else
  {
    end += 4;
  }
}
```

What is the value of `end` after the code segment is executed?

FINAL Unit2 MCQ - Topic order

(A)

5

(B) ✓

6

(C)

9

(D)

10

Answer B

Correct. The condition in the first outer `if` statement evaluates to `true` because `start < end` evaluates to `true`. In the first nested `if` statement, `end > 0` evaluates to `true`, so `start` and `end` are both assigned the value `6`. In the second outer `if` statement, `start < end` evaluates to `false` because the two variables now contain the same value; therefore, no further increments are done and the final value of `end` is `6`.

FINAL Unit2 MCQ - Topic order

40. Consider the following code segment.

```
int x = 7;
int y = 4;
boolean a = false;
boolean b = false;

if (x > y)
{
  if (x % y >= 3)
  {
    a = true;
    x -= y;
  }
  else
  {
    x += y;
  }
}

if (x < y)
{
  if (y % x >= 3)
  {
    b = true;
    x -= y;
  }
  else
  {
    x += y;
  }
}
```

What are the values of `a`, `b`, and `x` after the code segment is executed?

(A) `a = true, b = true, x = -1`

(B) `a = true, b = false, x = 3`

(C) `a = true, b = false, x = 7` ✓

(D) `a = false, b = false, x = 11`

Answer C

FINAL Unit2 MCQ - Topic order

Correct. The first three conditions encountered in the code segment evaluate to `true`. In the first `if` statement, since `x > y` and `x % y >= 3`, the body of the nested `if` clause is executed; `a` is assigned the value `true` and `x` is decremented by `y`, resulting in `3`. In the third `if` statement, since `x < y` but `y % x` is not `>= 3`, the body of the nested `else` clause is executed; `b` is unchanged and `x` is incremented by `y`, resulting in `7`. The variables `a`, `b`, and `x` are assigned the values `true`, `false`, and `7`, respectively.

FINAL Unit2 MCQ - Topic order

41. Consider the following code segment, which uses properly declared and initialized int variables x and y and the String variable result.

```
String result = "";
if (x < 5)
{
  if (y > 0)
  {
    result += "a";
  }
  else
  {
    result += "b";
  }
}
else if (x > 10)
{
  if (y < 0)
  {
    result += "c";
  }
  else if (y < 10)
  {
    result += "d";
  }
  result += "e";
}
result += "f";
```

What is the value of result after the code segment is executed if x has the value 15 and y has the value 5?

(A)

adf

(B)

d

(C) ✓

def

(D)

ef

FINAL Unit2 MCQ - Topic order

Answer C

Correct. The value of result will contain neither "a" nor "b" because x < 5 is false. Since x > 10 is true but y < 0 is false, the value of result does not contain "c". Since x > 10 is true and y < 10 is true, the value of result contains "d". The value of result contains "e" because the code line result += "e"; is outside the inner if-else structure. The value of result contains "f" because the code line result += "f"; is outside the outer if-else structure.

FINAL Unit2 MCQ - Topic order

42. Vehicles are classified based on their total interior volume. The `classify` method is intended to return a vehicle classification `String` value based on total interior volume, in cubic feet, as shown in the table below.

Vehicle size class	Total interior volume
Minicompact	Less than 85 cubic feet
Subcompact	85 to 99 cubic feet
Compact	100 to 109 cubic feet
Mid-Size	110 to 119 cubic feet
Large	120 cubic feet or more

The `classify` method, which does not work as intended, is shown below.

```
public static String classify(int volume)
{
 String carClass = "";
 if (volume >= 120)
 {
   carClass = "Large";
 }
 else if (volume < 120)
 {
   carClass = "Mid-Size";
 }
 else if (volume < 110)
 {
   carClass = "Compact";
 }
 else if (volume < 100)
 {
   carClass = "Subcompact";
 }
 else
 {
   carClass = "Minicompact";
 }
 return carClass;
}
```

The `classify` method works as intended for some but not all values of the parameter `volume`. For which of the following values of `volume` would the correct value be returned when the `classify` method is executed?

FINAL Unit2 MCQ - Topic order

(A) `80`
(B) `90`
(C) `105`
(D) `115` ✓

Answer D

FINAL Unit2 MCQ - Topic order

43. Consider the following code segment.

```
if (false && true || false)
{
  if (false || true && false)
  {
    System.out.print("First");
  }
  else
  {
    System.out.print("Second");
  }
}

if (true || true && false)
{
  System.out.print("Third");
}
```

What is printed as a result of executing the code segment?

(A)

Second

(B) ✓

Third

(C)

FirstThird

(D)

SecondThird

Answer B

Correct. The condition in the first `if` statement evaluates to `false`, so the nested `if` statement is not executed. The condition in the final `if` statement evaluates to `true`, so the string `"Third"` is printed.

FINAL Unit2 MCQ - Topic order

44. Consider the following method.

```
public void conditionalTest(int a, int b)
{
  if ((a > 0) && (b > 0))
  {
    if (a > b)
      System.out.println("A");
    else
      System.out.println("B");
  }
  else if ((b < 0) || (a < 0))
    System.out.println("C");
  else
    System.out.println("D");
}
```

What is printed as a result of the call `conditionalTest(3, -2)`?

(A) `A`

(B) `B`

(C) `C` ✓

(D) `D`

FINAL Unit2 MCQ - Topic order

45. In the following code segment, num is a properly declared and initialized int variable.

```
boolean b1 = true;
if (num > 0)
{
  if (num >= 100)
  {
    b1 = false;
  }
}
else
{
  if (num >= -100)
  {
    b1 = false;
  }
}
```

Which of the following statements always assigns the same value to b2 as the code segment assigns to b1?

(A)

```
boolean b2 = (num > -100) && (num < 100);
```

(B)

```
boolean b2 = (num > -100) || (num < 100);
```

(C)

```
boolean b2 = (num < -100) && (num > 0 || num < 100);
```

(D) ✓

```
boolean b2 = (num < -100) || (num > 0 && num < 100);
```

Answer D

Correct. In the body of the first if statement, b1 retains the value true if num is between 0 and 100, exclusive. In the body of the else statement, b1 retains the value true if num is less than -100. The statement assigns true to b2 if num is less than -100 or between 0 and 100, exclusive.

FINAL Unit2 MCQ - Topic order

46. The following incomplete method is intended to return the minimum value among its three parameters.

```
public static int minOfThree(int a, int b, int c)
{
  /* missing code */
}
```

Which of the following can be used to replace /* missing code */ so that the method works as intended?

FINAL Unit2 MCQ - Topic order

(A)

```
if (a <= b)
{
  if (a <= c)
  {
    return a;
  }
  else
  {
    return c;
  }
}
else
{
  return b;
}
```

FINAL Unit2 MCQ - Topic order

(B)

```
if (a <= b)
{
  if (a <= c)
  {
    return a;
  }
  else if (b <= c)
  {
    return b;
  }
}
else
{
  return c;
}
```

FINAL Unit2 MCQ - Topic order

(C)

```
if (a <= b || a <= c)
{
  return a;
}
else if (b <= a || b <= c)
{
  return b;
}
else
{
  return c;
}
```

FINAL Unit2 MCQ - Topic order

(D)

```
if (a <= b && a <= c)
{
  return a;
}
else if (b <= a && b <= c)
{
  return b;
}
else
{
  return c;
}
```

Answer D

Correct. In the if clause, a is returned if it is less than or equal to the other two variables, which means it is the minimum. In the if else clause, b is returned if it is less than or equal to the other two variables, which means it is the minimum. In the else clause, c is returned when it is determined that the other two variables are not the minimum.

FINAL Unit2 MCQ - Topic order

47. In the following code segment, num is a properly declared and initialized int variable.

```
if (num > 0)
{
  if (num % 2 == 0)
  {
    System.out.println("X");
  }
  else
  {
    System.out.println("Y");
  }
}
```

Which of the following best describes the behavior of the code segment?

(A) When num is a negative odd integer, "X" is printed; otherwise, "Y" is printed.

(B) When num is a positive even integer, "X" is printed; otherwise, "Y" is printed.

(C) When num is a positive even integer, "X" is printed; when num is a positive odd integer, "Y" is printed; otherwise, nothing is printed. ✓

(D) When num is a positive odd integer, "X" is printed; when num is a positive even integer, "Y" is printed; otherwise, nothing is printed.

Answer C

Correct. When num is positive and even, the conditions in the outer and inner if statements are true, so "X" is printed. When num is positive and not even (odd), the condition in the outer if statement is true and the condition in the the inner if statement is false, so "Y" is printed. When num is not positive, the condition in the outer if statement is false, so nothing is printed.

FINAL Unit2 MCQ - Topic order

48. A fitness application categorizes daily step counts into different activity levels. The "Active" level is defined as any step count between 10,000 and 15,000, inclusive. The following code segment is intended to determine if a user's step count falls within the "Active" category.

```
if (steps > 10000 && steps < 15000)
{
  activityLevel = "Active";
}
```

Which of the following best explains the error, if any, in this code segment?

(A) There is no error in the code segment as it correctly identifies "Active" step counts.

(B) The condition should be `if (steps >= 10000 && steps <= 15000)` to properly include 10,000 and 15,000. ✓

(C) The condition should be `if (steps > 10000 || steps < 15000)` to include the correct range of step counts.

(D) The condition should be `if (steps >= 10000 || steps <= 15000)` to include all step counts from 10,000 and above, as well as 15,000 and below.

Answer B

Correct. This solution correctly checks if `steps` is greater than or equal to 10,000 and less than or equal to 15,000, matching the "Active" category definition.

FINAL Unit2 MCQ - Topic order

49. A certain device requires a passcode to be used. The passcode must meet both of the following requirements.

- The passcode must be 4 digits long.
- The passcode must not start with a zero (0) digit.

Assume that pass is a properly declared and initialized int variable.

Which of the following expressions will evaluate to true if pass is an acceptable passcode?

(A)

```
pass >= 1000 || pass < 10000
```

(B)

```
pass > 1000 || pass <= 10000
```

(C) ✓

```
pass >= 1000 && pass < 10000
```

(D)

```
pass > 1000 && pass < 10000
```

Answer C

Correct. The first expression pass >= 1000 is false for values of pass with fewer than 4 digits, and the second expression pass < 10000 is false for values of pass with greater than 4 digits. The && (and) operator requires that both expressions be true.

FINAL Unit2 MCQ - Topic order

50. Consider the following code segment.

```
boolean a = true;
boolean b = true;
System.out.print((b || (!a || b)) + " ");
System.out.print(((!b || !a) && a) + " ");
System.out.println(!(a && b) && b);
```

What output is produced when this code segment is executed?

(A)

true true true

(B)

true false true

(C) ✓

true false false

(D)

false true false

Answer C

Correct. Since b is true, the first logical expression evaluates to true. The expression (!a || b) is not evaluated due to short-circuit evaluation. Since !b is false and !a is false, (!b || !a) evaluates to false, and the rest of the expression is not evaluated due to short-circuit evaluation. Since (a && b) evaluates to true, !(a && b) evaluates to false, and the third logical expression evaluates to false. The expression b is not evaluated due to short-circuit evaluation.

FINAL Unit2 MCQ - Topic order

51. Consider the following `boolean` variables.

- `isRaining` is `true` if it is raining and is `false` otherwise.
- `isSunny` is `true` if it is sunny and is `false` otherwise.
- `isWarm` is `true` if it is warm and is `false` otherwise.

Assume all three variables have been properly declared and initialized.

Which of the following expressions evaluates to `true` if it is sunny or warm and it is not raining, and evaluates to `false` otherwise?

(A)

(isSunny || isWarm) && isRaining

(B) ✓

(isSunny || isWarm) && !isRaining

(C)

!(isSunny || isWarm) || isRaining

(D)

!(isSunny || isWarm) || !isRaining

Answer B

Correct. If it is sunny or warm or both sunny and warm, then the expression `(isSunny || isWarm)` evaluates to `true`. If it is not raining, then the expression `!isRaining` evaluates to `true`. So the entire expression evaluates to `true`. If it is not sunny and it is not warm, the expression `(isSunny || isWarm)` evaluates to `false`, and the whole expression is `false`. If it is raining, the expression `!isRaining` evaluates to `false`, and the whole expression is `false`.

FINAL Unit2 MCQ - Topic order

52. Which of the following best describes the value of the Boolean expression shown below?

```
a && !(b || a)
```

(A) The value is always `false`. ✓

(B) The value is `true` when `a` has the value `false`, and is `false` otherwise.

(C) The value is `true` when `b` has the value `false`, and is `false` otherwise.

(D) The value is `true` when either `a` or `b` has the value `true`, and is `false` otherwise.

Answer A

53. Assume that x, y, and z are boolean variables that have been properly declared and initialized.

Which of the following expressions is equivalent to !x || !y || z?

(A)

!(x && y) && z

(B) ✓

!(x && y) || z

(C)

!(x || y) && z

(D)

!(x || y) || z

Answer B

Correct. By De Morgan's law, !x || !y is equivalent to !(x && y) and the entire expression is equivalent to !(x && y) || z.

FINAL Unit2 MCQ - Topic order

54. Assume that isEven, isPositive, and isPrime are boolean variables that have been properly declared and initialized. Which of the following expressions is equivalent to the expression ! (isEven && isPositive) && isPrime?

(A) ✓

(!isEven || !isPositive) && isPrime

(B)

(!isEven || !isPositive) && !isPrime

(C)

(!isEven && !isPositive) && isPrime

(D)

(!isEven && !isPositive) && !isPrime

Answer A

Correct. By De Morgan's law, !(isEven && isPositive) is equivalent to !isEven || !isPositive. The entire expression is equivalent to (!isEven || !isPositive) && isPrime.

55. Consider the following code segment.

```
int num1 = 0;
int num2 = 3;
while ((num2 != 0) && ((num1 / num2) >= 0))
{
  num1 = num1 + 2;
  num2 = num2 - 1;
}
```

What are the values of `num1` and `num2` after the while loop completes its execution?

(A) `num1 = 0, num2 = 3`

(B) `num1 = 8, num2 = -1`

(C) `num1 = 4, num2 = 1`

(D) `num1 = 6, num2 = 0` ✓

FINAL Unit2 MCQ - Topic order

56. Consider the following method.

```
public void numberCheck(int maxNum)
{
  int typeA = 0;
  int typeB = 0;
  int typeC = 0;

  for (int k = 1; k <= maxNum; k++)
  {
    if (k % 2 == 0 && k % 5 == 0)
     typeA++;
    if (k % 2 == 0)
      typeB++;
    if (k % 5 == 0)
      typeC++;
  }

  System.out.println(typeA + " " + typeB + " " + typeC);
}
```

What is printed as a result of the call `numberCheck(50)` ?

(A) `5 20 5`

(B) `5 20 10`

(C) `5 25 5`

(D) `5 25 10` ✓

FINAL Unit2 MCQ - Topic order

57. Consider the following code segment.

```
String str = "a black cat sat on a table";
int counter = 0;

for (int i = 0; i < str.length() - 1; i++)
{
  if (str.substring(i, i + 1).equals("a") &&
      !str.substring(i + 1, i + 2).equals("b"))
  {
    counter++;
  }
}

System.out.println(counter);
```

What is printed as a result of executing the code segment?

(A)

1

(B)

2

(C)

3

(D) ✓

5

Answer D

Correct. The expression str.substring(i, i + 1).equals("a") will evaluate to true if str contains the string "a" at position i. The expression !str.substring(i + 1, i + 2).equals("b") will evaluate to true if str does not contain the string "b" at position i + 1. The variable counter is incremented when both of those expressions evaluate to true or, in other words, whenever "a" is not immediately followed by "b". This occurs five times in the given string: "a ", "ac", "at", "at", and "a ".

FINAL Unit2 MCQ - Topic order

58. Consider the following code segment.

```
String str1 = new String("Advanced Placement");
String str2 = new String("Advanced Placement");
if (str1.equals(str2) && str1 == str2)
{
 System.out.println("A");
}
else if (str1.equals(str2) && str1 != str2)
{
 System.out.println("B");
}
else if (!str1.equals(str2) && str1 == str2)
{
 System.out.println("C");
}
else if (!str1.equals(str2) && str1 != str2)
{
 System.out.println("D");
}
```

What, if anything, is printed when the code segment is executed?

(A) `A`

(B) `B` ✓

(C) `C`

(D) `D`

Answer B

Correct. Both `str1` and `str2` contain the string `"Advanced Placement"`, so the `equals` method returns `true`. Comparing `str1` and `str2` with `!=` returns `true` since the two `String` objects are declared and instantiated with separate calls to the `String` constructor and are not the same object.

FINAL Unit2 MCQ - Topic order

59. Consider the following class definition.

```
public class Backyard
{
  private int length;
  private int width;

  public Backyard(int l, int w)
  {
    length = l;
    width = w;
  }

  public int getLength()
  {
    return length;
  }

  public int getWidth()
  {
    return width;
  }

  public boolean equals(Object other)
  {
    if (other == null)
    {
      return false;
    }

    Backyard b = (Backyard) other;
    return (length == b.getLength() && width == b.getWidth());
  }
}
```

The following code segment appears in a class other than `Backyard`. It is intended to print `true` if `b1` and `b2` have the same lengths and widths, and to print `false` otherwise. Assume that `x`, `y`, `j`, and `k` are properly declared and initialized variables of type `int`.

```
Backyard b1 = new Backyard(x, y);
Backyard b2 = new Backyard(j, k);
System.out.println( /* missing code */ );
```

Which of the following can be used as a replacement for `/* missing code */` so the code segment works as intended?

FINAL Unit2 MCQ - Topic order

(A) `b1.equals(b2)` ✓

(B) `equals(b1, b2)`

(C) `b1.equals(b2.getLength(), b2.getWidth())`

(D) `b1.length == b2.length && b1.width == b2.width`

Answer A

Correct. In the `equals` method, `length` and `width` refer to the instance data of the object `b1`. The formal parameter `b` refers to the object `b2` based on the actual parameter in the method call. The `if` statement in the `equals` method therefore compares the length and width of `b1` with the length and width of `b2`.

FINAL Unit2 MCQ - Topic order

60. In the following Boolean expressions, `x` and `y` are properly declared and initialized `boolean` variables.

Expression A

x && y

Expression B

!x && !y

Which of the following best describes the relationship between values produced by Expression A and Expression B?

(A) Expression A and Expression B evaluate to different values for all values of `x` and `y`.

(B) Expression A and Expression B evaluate to the same value for all values of `x` and `y`.

(C) Expression A and Expression B evaluate to the same value only when `x` and `y` have the same value.

(D) Expression A and Expression B evaluate to the same value only when `x` and `y` have different values. ✓

Answer D

Correct. The following truth table can be used to show that both expressions evaluate to the same value (`false`) when `x` and `y` have different values and that they evaluate to different values when `x` and `y` have the same value.

x	y	x && y	!x && !y
true	true	true	false
true	false	false	false
false	true	false	false
false	false	false	true

FINAL Unit2 MCQ - Topic order

61. Assume `obj1` and `obj2` are object references. Which of the following best describes when the expression `obj1 == obj2` is true?

(A) When `obj1` and `obj2` are defined within the same method

(B) When `obj1` and `obj2` are instances of the same class

(C) When `obj1` and `obj2` refer to objects that contain the same data

(D) When `obj1` and `obj2` refer to the same object ✓

62. Assume that `x` and `y` are `boolean` variables and have been properly initialized.

```
(x && y) || !(x && y)
```

The result of evaluating the expression above is best described as

(A) always `true` ✓

(B) always `false`

(C) `true` only when `x` is `true` and `y` is `true`

(D) `true` only when `x` and `y` have the same value

FINAL Unit2 MCQ - Topic order

63. In the following code segment, x is a properly declared and initialized boolean variable and y is a properly declared and initialized int variable.

```
boolean a = !x && y > 50000;
boolean b = !(x || y <= 50000);
boolean c = !x || y <= 50000;
boolean d = !(x && y <= 50000);

if (a == b)
{
  System.out.println("First");
}
if (a == c)
{
  System.out.println("Second");
}
if (b == c)
{
  System.out.println("Third");
}
if (c == d)
{
  System.out.println("Fourth");
}
```

Which of the following is always printed as a result of executing this code segment?

(A) ✓

First

(B)

Second

(C)

Third

(D)

Fourth

Answer A

Correct. Applying De Morgan's law to the definition of b yields !x && y > 50000. This is the same as a.

64. Consider the following code segment.

```
String alpha = new String("APCS");
String beta = new String("APCS");
String delta = alpha;
System.out.println(alpha.equals(beta));
System.out.println(alpha == beta);
System.out.println(alpha == delta);
```

What is printed as a result of executing the code segment?

(A)
```
false
false
true
```

(B)
```
true
false
false
```

(C) ✓
```
true
false
true
```

(D)
```
true
true
true
```

Answer C

Correct. The `equals` method is used to determine if two objects have the same content. Since both `alpha` and `beta` contain the same sequence of characters, the first `print` statement prints `true`. The `==` operator is used to compare object references. Since `alpha` and `beta` are not the same object, the second `print` statement prints `false`. Since `delta` is initialized to `alpha`, `delta` and `alpha` contain the same object reference and the third `print` statement prints `true`.

FINAL Unit2 MCQ - Topic order

65. Assume that object references `one`, `two`, and `three` have been declared and instantiated to be of the same type. Assume also that `one == two` evaluates to `true` and that `two.equals(three)` evaluates to `false`.

Consider the following code segment.

```
if (one.equals(two))
{
 System.out.println("one dot equals two");
}
if (one.equals(three))
{
 System.out.println("one dot equals three");
}
if (two == three)
{
 System.out.println("two equals equals three");
}
```

What, if anything, is printed as a result of executing the code segment?

(A) `one dot equals two` ✓

(B)
```
one dot equals two
one dot equals three
```

(C)
```
one dot equals three
two equals equals three
```

(D) Nothing is printed.

Answer A

Correct. The `equals` method is used to determine if two objects have the same content, and the `==` operator is used to compare object references. Since `one == two` evaluates to `true`, `one` and `two` must reference the same object and the content of `one` must be the same as the content of `two`. The condition in the first `if` statement evaluates to `true`, and `"one dot equals two"` is printed. The content of `one` is the same as the content of `two`, but the content of `two` is not the same as the content of `three`. Therefore the content of `one` cannot be the same as the content of `three`. The condition in the second `if` statement evaluates to `false`, and the `print` statement in the body of this `if` statement is not executed. If the content of `two` is not the same as the content of `three`, then their object references cannot be the same. The condition in the third `if` statement evaluates to `false`, and the `print` statement in the body of this `if` statement is also not executed.

FINAL Unit2 MCQ - Topic order

66. Assume that a, b, and c are boolean variables that have been properly declared and initialized. Which of the following boolean expressions is equivalent to !(a && b) || c?

(A)

a || b || c

(B)

!a && !b || c

(C)

!a && !b && c

(D) ✓

!a || !b || c

Answer D

Correct. By De Morgan's law, !(a && b) is equivalent to !a || !b and the entire expression is equivalent to !a || !b || c.

67. Assume that `a`, `b`, `c`, and `d` have been declared and initialized with `int` values.

```
!((a >= b) && !(c < d))
```

Which of the following is equivalent to the expression above?

(A) `(a < b) || (c < d)` ✓

(B) `(a < b) || (c >= d)`

(C) `(a < b) && (c < d)`

(D) `(a >= b) || (c < d)`

Answer A

Correct. By De Morgan's laws, the given expression is equivalent to `(a < b) || (c < d)`.

FINAL Unit2 MCQ - Topic order

68. An image on a computer is made up of many pixels. Each pixel in the image contains varying amounts of red, green, and blue. A "mainly red" pixel is defined to be one in which the percentage of red is more than 50, and the percentages of green and blue are each less than 10.

Assume that redPct, greenPct, and bluePct are int variables that have been properly declared and initialized with values representing the percentages of red, green, and blue, respectively, in a pixel.

Which of the following expressions will evaluate to true for a "mainly red" pixel?

(A)

(redPct <= 50) || (greenPct >= 10) || (bluePct >= 10)

(B)

(redPct > 50) && (greenPct < 10) || (bluePct < 10)

(C) ✓

!((redPct <= 50) || (greenPct >= 10) || (bluePct >= 10))

(D)

!(redPct > 50) && !(greenPct > 10) && !(bluePct > 10)

Answer C

Correct. Application of De Morgan's law yields the code segment if (redPct > 50 && greenPct < 10 && bluePct < 10) which correctly identifies "mainly red" pixels.

69. Assume that `a` and `b` are variables of type `int`. The expression

```
!(a < b) && !(a > b)
```

is equivalent to which of the following?

(A) `true`

(B) `false`

(C) `a == b` ✓

(D) `a != b`

FINAL Unit2 MCQ - Topic order

70. Consider the following class definition.

```
public class Time
{
 private int hours;
 private int minutes;

 public Time(int h, int m)
 {
   hours = h;
   minutes = m;
 }

/** Returns true if hours * 60 + minutes of this
  *     object is equal to hours * 60 + minutes of object
   *   other; otherwise returns false
  */
 public boolean equals(Object other)
   {
     //implementation not shown
  }
}
```

The following code segment appears in a class other than `Time`.

```
Time t1 = new Time(1, 10);
Time t2 = new Time(0, 70);
```

Which of the following statements will print `true` ?

I. `System.out.println(t1 == t2);`
II. `System.out.println(t1.equals(t2));`
III. `System.out.println(equals(t1, t2);`

(A) I only
(B) II only ✓
(C) III only
(D) I and II

Answer B

FINAL Unit2 MCQ - Topic order

Correct. Statement I will print `false` because `t1` and `t2` refer to different objects. Statement II will print `true`. The `equals` method compares `t1` (1 hour and 10 minutes) to `t2` (70 minutes), evaluating to `true`. Statement III causes a compile-time error because the `equals` method must be called on an object and only takes one parameter.

71. Consider the following code segment. The code is intended to read nonnegative numbers and compute their product until a negative number is read; however, it does not work as intended. (Assume that the `readInt` method correctly reads the next number from the input stream.)

```
int k = 0;
int prod = 1;

while (k >= 0)
{
  System.out.print("enter a number: ");
  k = readInt(); // readInt reads the next number from input
  prod = prod * k;
}

System.out.println("product: " + prod);
```

Which of the following best describes the error in the program?

(A) The variable `prod` is incorrectly initialized.

(B) The while condition always evaluates to `true`.

(C) The negative number entered to signal no more input is included in the product. ✓

(D) If the user enters a zero, the computation of the product will be terminated prematurely.

FINAL Unit2 MCQ - Topic order

72. Consider the following code segment.

```
int count = 5;
while (count < 100)
{
  count = count * 2;
}
count = count + 1;
```

What is the value of `count` after executing the code segment?

(A)

100

(B)

101

(C)

160

(D) ✓

161

Answer D

Correct. The initial value of `count` is `5`. The `while` loop repeatedly doubles the value of `count` until it is at least `100`. The successive values of `count` are `10`, `20`, `40`, `80`, and `160`. When `count` reaches `160` (the first value greater than or equal to `100`), the loop terminates. The last line of code increases `count` by `1`, resulting in `161`.

FINAL Unit2 MCQ - Topic order

73. Consider the following code segment.

```
int value = 15;
while (value < 28)
{
  System.out.println(value);
  value++;
}
```

What are the first and last numbers output by the code segment?

(A) ✓

First	Last
15	27

(B)

First	Last
15	28

(C)

First	Last
16	27

(D)

First	Last
16	28

FINAL Unit2 MCQ - Topic order

74. Consider the following code segment.

```
int count = 0;
for (int k = 0; k < 10; k++)
{
  count++;
}
System.out.println(count);
```

Which of the following code segments will produce the same output as the code segment above?

(A)
```
int count = 0;
for (int k = 1; k < 10; k++)
{
  count++;
}
System.out.println(count);
```

(B)
```
int count = 1;
for (int k = 1; k <= 10; k++)
{
  count++;
}
System.out.println(count);
```

(C)
```
int count = 1;
for (int k = 0; k <= 9; k++)
{
  count++;
}
System.out.println(count);
```

(D) ✓
```
int count = 0;
for (int k = 9; k >= 0; k--)
{
  count++;
}
System.out.println(count);
```

Answer D

Correct. The loop iterates 10 times. In each iteration, `count` is incremented by `1`. Since the initial value of `count` is `0`, the printed value of `count` is `10`.

FINAL Unit2 MCQ - Topic order

75. In the following code segment, `numOne` and `numTwo` are properly declared and initialized `int` variables.

```
while (numOne > 1)
{
  if (numOne % numTwo == 0)
  {
    numOne = numOne / numTwo;
  }
}
```

Which of the following values for `numOne` and `numTwo` causes an infinite loop to occur when the code segment is executed?

(A) `numOne = 16 and numTwo = 2`

(B) `numOne = 100 and numTwo = 10`

(C) `numOne = 125 and numTwo = 5`

(D) `numOne = 128 and numTwo = 4` ✓

Answer D

Correct. An infinite loop occurs when `numOne` is not a power of `numTwo`. In the first iteration, `numOne % numTwo` evaluates to `true` and `numOne` is assigned the value `32`. In the second iteration of the loop, `numOne % numTwo` evaluates to `true` and `numOne` is assigned the value `8`. In the third iteration of the loop, `numOne` is assigned the value `2`. In the fourth iteration, `numOne % numTwo` evaluates to `false` and `numOne` is not updated. This causes an infinite loop since `numOne` is never updated again. The Boolean expression `numOne > 1` always evaluates to `true` so the loop never terminates.

FINAL Unit2 MCQ - Topic order

76. Which of the following code segments produces the output "987654321"?

FINAL Unit2 MCQ - Topic order

(A)

```
int num = 10;
while (num > 0)
{
  System.out.print(num);
  num--;
}
```

(B)

```
int num = 10;
while (num > 1)
{
  System.out.print(num);
  num--;
}
```

(C) ✓

```
int num = 10;
while (num > 1)
{
  num--;
  System.out.print(num);
}
```

FINAL Unit2 MCQ - Topic order

(D)

```
int num = 10;

while (num >= 1)

{

  num--;

  System.out.print(num);

}
```

Answer C

Correct. During the first iteration of the while loop, num is decremented and "9" is printed. During the second iteration, num is decremented and "8" is printed. This continues until the last iteration of the loop, when num is decremented and "1" is printed. At this point, the Boolean expression in the while loop evaluates to false and the loop terminates.

FINAL Unit2 MCQ - Topic order

77. Consider the following code segment.

```
int k = 0;
/* missing loop header */
{
  k++;
  System.out.print(k + " ");
}
```

Which of the following can be used as a replacement for `/* missing loop header */` so that the code segment prints out the string `"1 2 3 4 "`?

(A) `while (k < 4)` ✓

(B) `while (k < 5)`

(C) `while (k <= 4)`

(D) `while (k <= 5)`

Answer A

Correct. The loop is entered when `k` has the values `0` through `3`, inclusive. In each case, `k` is incremented and then printed followed by a space, producing the string `"1 2 3 4 "`.

FINAL Unit2 MCQ - Topic order

78. Consider the following code segment.

```
int num = 1;
while (num < 5)
{
 System.out.print("A");
 num += 2;
}
```

What is printed as a result of executing the code segment?

(A) `A`

(B) `AA` ✓

(C) `AAA`

(D) `AAAA`

Answer B

Correct. The string is printed two times because the body of the `while` loop is executed twice. The variable `num` is initialized to `1`, so the Boolean expression `num < 5` evaluates to `true`, and the loop body executes. The string `"A"` is printed, then `num` is reassigned to `num + 2`, or `3`. The Boolean expression `num < 5` still evaluates to `true`, and the loop body executes a second time. The string `"A"` is printed (on the same line), then `num` is reassigned to `num + 2`, or `5`. The Boolean expression `num < 5` now evaluates to `false`, so the loop is exited and the code segment completes it execution.

FINAL Unit2 MCQ - Topic order

79. The following code segment is intended to print the number of digits in the `int` variable `num`, where `num > 0`.

```
int count = 0;
while ( /* missing condition */ )
{
  count++;
  num = num / 10;
}
System.out.println(count);
```

Which of the following can be used to replace `/* missing condition */` so that the code segment works as intended?

(A)

count != 0

(B)

count > 0

(C)

num >= 0

(D) ✓

num != 0

Answer D

Correct. The value of `num` is repeatedly divided by `10`. Each time `num` is divided by `10`, it will result in an integer that has one fewer digit than `num`, and `count` will be incremented by `1`. This process will continue until `num` eventually reaches `0`, at which point the loop will terminate. As part of the the process, the variable `count` will have been incremented by `1` in the loop for each digit of `num`.

FINAL Unit2 MCQ - Topic order

80. In the following code segment, x is an int variable with a positive value.

```
int temp = x;
while (temp > 0)
{
  temp -= 2;
}
System.out.println(temp == 0);
```

Which of the following best describes the behavior of the code segment?

(A) It prints 0 if x is even and prints -1 otherwise.

(B) It prints -1 if x is even and prints 0 otherwise.

(C) It prints true if x is even and prints false otherwise. ✓

(D) It prints false if x is even and prints true otherwise.

Answer C

Correct. The code segment assigns the value of x to temp, then repeatedly subtracts 2 from temp until temp is no longer greater than 0. At this point, temp will be either 0 or -1. If temp is 0, x must be even because it can be obtained by repeatedly adding up the value 2. When temp is 0, the expression temp == 0 evaluates to true.

FINAL Unit2 MCQ - Topic order

81. Consider the following code segment.

```
int a = 1;
String result = "";
while (a < 20)
{
  result += a;
  a += 5;
}
System.out.println(result);
```

What is printed as a result of executing this code segment?

(A)

21

(B) ✓

161116

(C)

161161

(D)

16111621

Answer B

Correct. The code segment prints 161116. In the first iteration of the while loop, the string "1" is appended to result and the value of a is set to 6. In the second iteration, the string "6" is appended to result and the value of a is set to 11. In the third iteration, the string "11" is appended to result and the value of a is set to 16. In the fourth (and last) iteration, the string "16" is appended to result, the value of a is set to 21, and the loop terminates because the condition a < 20 is no longer true.

FINAL Unit2 MCQ - Topic order

82. Which of the following code segments can be used to print only the even integers between 2 and 10, inclusive?

FINAL Unit2 MCQ - Topic order

(A)

```
int num = 0;
while(num < 10)
{
  System.out.println(num);
  num += 2;
}
```

(B)

```
int num = 0;
while(num <= 10)
{
  System.out.println(num);
  num += 2;
}
```

(C) ✓

```
int num = 0;
while(num < 10)
{
  num += 2;
  System.out.println(num);
}
```

FINAL Unit2 MCQ - Topic order

(D)

```
int num = 0;
while(num <= 10)
{
  num += 2;
  System.out.println(num);
}
```

Answer C

Correct. This code segment initializes num to 0. Inside the loop, num is incremented by 2 and then printed. This continues until num is incremented from 8 to 10, and then the loop terminates.

FINAL Unit2 MCQ - Topic order

83. Consider the following code segment.

```
int j = 0;
int sum = 0;

while (j <= 5)
{
  j++;
  sum += j;
}
```

Which of the following code segments assigns the same value to sum as the preceding code segment?

FINAL Unit2 MCQ - Topic order

(A)

```
int sum = 0;
for (int j = 0; j <= 5; j++)
{
  j++;
  sum += j;
}
```

(B)

```
int sum = 0;
for (int j = 0; j <= 6; j++)
{
  j++;
  sum += j;
}
```

(C)

```
int sum = 0;
for (int j = 1; j <= 5; j++)
{
  sum += j;
}
```

FINAL Unit2 MCQ - Topic order

(D)

```
int sum = 0;

for (int j = 1; j <= 6; j++)

{

  sum += j;

}
```

Answer D

Correct. The given while loop calculates the sum of the integers from 1 to 6. It iterates six times (from j = 0 to j = 5), and increments j just before adding it to sum. This for loop also calculates the sum of the integers from 1 to 6. It iterates six times (from j = 1 to j = 6), adding each value to sum.

FINAL Unit2 MCQ - Topic order

84. Consider the following code segment.

```
int val = 48;
int div = 6;
while ((val % 2 == 0) && div > 0)
{
 if (val % div == 0)
 {
   System.out.print(val + " ");
 }
 val /= 2;
 div--;
}
```

What is printed when the code segment is executed?

(A) `48 12 6` ✓

(B) `48 12 6 3`

(C) `48 12 6 3 1`

(D) `48 24 12 6`

Answer A

FINAL Unit2 MCQ - Topic order

85. Consider the following code segment, which is intended to print the sum of all the odd integers from 1 to 101, inclusive.

```
int r = 1;
int sum = 0;

while (r <= 101)
{
  /* missing code */
  r++;
}
System.out.println(sum);
```

Which of the following can be used to replace `/* missing code */` so that the code segment works as intended?

FINAL Unit2 MCQ - Topic order

(A)

```
if (r % 2 != 1)
{
  sum += r;
}
```

(B)

```
if (r % 2 != 1)
{
  sum++;
}
```

(C) ✓

```
if (r % 2 == 1)
{
  sum += r;
}
```

(D)

```
if (r % 2 == 1)
{
  sum++;
}
```

Answer C

Correct. The condition in the `if` statement evaluates to `true` when `r` is odd. The body of the `while` loop

adds `r` to `sum` to calculate the sum of the odd integers between 1 and 101, inclusive.

86. Consider the following method definition. The method `printAllCharacters` is intended to print out every character in `str`, starting with the character at index `0`.

```
public static void printAllCharacters(String str)
{
 for (int x = 0; x < str.length(); x++)   // Line 3
 {
   System.out.print(str.substring(x, x + 1));
 }
}
```

The following statement is found in the same class as the `printAllCharacters` method.

```
printAllCharacters("ABCDEFG");
```

Which choice best describes the difference, if any, in the behavior of this statement that will result from changing `x < str.length()` to `x <= str.length()` in line 3 of the method?

(A) The method call will print fewer characters than it did before the change because the loop will iterate fewer times.

(B) The method call will print more characters than it did before the change because the loop will iterate more times.

(C) The method call, which worked correctly before the change, will now cause a run-time error because it attempts to access a character at index `7` in a string whose last element is at index `6`. ✓

(D) The method call, which worked correctly before the change, will now cause a run-time error because it attempts to access a character at index `8` in a string whose last element is at index `7`.

Answer C

Correct. With the original condition, method call prints `"ABCDEFG"`. With the new condition, the method call causes the loop to iterate one additional time when `k = 7`, causing a run-time error because there is no character at index `7`.

FINAL Unit2 MCQ - Topic order

87. Consider the following method.

```
/** Precondition: bound >= 0 */
public int sum(int bound)
{
 int answer = 0;
 for (int i = 0; i < bound; i++)
 {
   answer += bound;
 }
 return answer;
}
```

Assume that `sum` is called with a parameter that satisfies the precondition and that it executes without error. How many times is the test expression `i < bound` in the `for` loop header evaluated?

(A) `bound - 1`

(B) `bound`

(C) `bound + 1` ✓

(D) An unknown number of times

Answer C

Correct. When `bound` is `0`, the Boolean expression in the `for` loop header is evaluated once and is found to be `false`, so the loop body is never executed. When `bound` is `1`, the Boolean expression is evaluated twice—the first time it evaluates to `true` and the second time it evaluates to `false`. In general, the Boolean expression is evaluated `bound + 1` times—the first `bound` times, when it evaluates to `true` and the loop body is executed, and the last time, when it evaluates to `false` and the loop terminates.

FINAL Unit2 MCQ - Topic order

88. Consider the following two code segments. Code segment 2 is a revision of code segment 1 in which the loop header has been changed.

Code Segment 1

```
for (int k = 1; k <= 5; k++)
{
  System.out.print(k);
}
```

Code Segment 2

```
for (int k = 5; k >= 1; k--)
{
  System.out.print(k);
}
```

Which of the following best explains the difference, if any, between the output of code segment 1 and code segment 2?

(A) Both code segments produce the same output, because they both iterate four times.

(B) Both code segments produce the same output, because they both iterate five times.

(C) Code segment 1 prints more values than code segment 2 does, because it iterates for one additional value of k.

(D) The code segments print the same values but in a different order, because code segment 1 iterates from 1 to 5 and code segment 2 iterates from 5 to 1. ✓

Answer D

Correct. Code segment 1 iterates from 1 to 5, producing the output 12345. Code segment 2 iterates from 5 to 1, producing the output 54321.

FINAL Unit2 MCQ - Topic order

89. Consider the following code segment.

```
String word = "computer";
int num = 3;
String result = "";

for (int k = num; k >= 0; k--)
{
   result += word.substring(0, k);
}

System.out.print(result);
```

What is printed as a result of executing the code segment?

(A) `comcoc` ✓

(B) `ccocomcomp`

(C) `compcomcoc`

(D) `comcomcomcom`

Answer A

Correct. The code segment prints the string `"comcoc"`. In the `for` loop, the integer variable `k` varies from `3` to `0`, so the loop executes four times. In the first loop iteration, `word.substring(0, 3)`, which is `"com"`, is appended to `result`. In the second loop iteration, `word.substring(0, 2)`, which is `"co"`, is appended to `result`. In the third loop iteration, `word.substring(0, 1)`, which is `"c"`, is appended to `result`. In the fourth loop iteration, `word.substring(0, 0)`, which is `""`, is appended to `result`.

FINAL Unit2 MCQ - Topic order

90. Consider the following code segment.

```
for (int j = 1; j < 10; j += 2)
{
  System.out.print(j);
}
```

Which of the following code segments will produce the same output as the given code segment?

FINAL Unit2 MCQ - Topic order

(A)

```
int j = 1;
while (j < 10)
{
  j += 2;
  System.out.print(j);
}
```

(B) ✓

```
int j = 1;
while (j < 10)
{
  System.out.print(j);
  j += 2;
}
```

(C)

```
int j = 1;
while (j <= 10)
{
  j += 2;
  System.out.print(j);
}
```

FINAL Unit2 MCQ - Topic order

(D)

```
int j = 1;

while (j >= 10)

{

  j += 2;

  System.out.print(j);

}
```

Answer B

Correct. In the given `for` loop, `j` is initially `1` and increases by `2` repeatedly as long as it is less than `10`. In this `while` loop, `j` is initially `1` and increases by `2` repeatedly while it is less than `10`. Both code segments produce the output `13579`.

FINAL Unit2 MCQ - Topic order

91. Consider the following code segment.

```
for (int k = 1; k <= 7; k += 2)
{
  System.out.print(k);
}
```

Which of the following code segments will produce the same output as the code segment above?

(A)
```
for (int k = 0; k < 7; k += 2)
{
  System.out.print(k);
}
```

(B)
```
for (int k = 0; k <= 8; k += 2)
{
  System.out.print(k + 1);
}
```

(C)
```
for (int k = 1; k < 7; k += 2)
{
  System.out.print(k + 1);
}
```

(D) ✓
```
for (int k = 1; k <= 8; k += 2)
{
  System.out.print(k);
}
```

Answer D

Correct. The given code segment starts when `k` is `1` and prints every other value as long as `k` is less than or equal to `7`. This code segment starts when `k` is `1` and prints every other value as long as `k` is less than or equal to `8`. Both code segments print `1357`.

FINAL Unit2 MCQ - Topic order

92. Consider the following two code segments, which are intended to produce identical outputs.

Code Segment 1

```
for (int i = 0; i < 10; i++)
{
  System.out.println("counting " + i);
}
```

Code Segment 2

```
int x = 0;
while (x < 10)
{
  x = x + 1;
  System.out.println("counting " + x);
}
```

Which of the following changes can be made to Code Segment 2 so that the outputs of the two code segments are identical as intended?

(A) Move the statement `x = x + 1;` so that it follows the print statement inside the loop body. ✓

(B) Rename the variable `x` to be `i` so that it is the same variable name as in Code Segment 1.

(C) Remove the statement `x = x + 1;` because this statement is not in the loop body in Code Segment 1.

(D) Change the statement `x = x + 1;` to be `x++;` so that the increment of the loop control variable is the same in both code segments.

Answer A

Correct. Moving `x = x + 1;` to the last statement to be executed in the `while` loop body will increment the loop control variable at the same point as the `for` loop in Code Segment 1.

FINAL Unit2 MCQ - Topic order

93. Consider the following code segment.

```
for (int j = 10; j >= 0; j -= 2)   // Line 1
{                                  // Line 2
  System.out.print(j – 1);         // Line 3
}                                  // Line 4
```

This code segment is intended to produce only the following output. It does not work as intended.

97531

Which of the following changes can be made so that this code segment works as intended?

(A) In line 1, changing the condition j >= 0 to j > 0 ✓

(B) In line 1, changing the initialization j = 10 to j = 9

(C) In line 3, changing the expression j - 1 to j + 1

(D) In line 3, changing the expression j - 1 to j - 2

Answer A

Correct. The given code segment prints "97531-1". It loops from j = 10 to j = 0, decrementing by two for each iteration. The body of the loop prints j - 1, so the extra value -1 is printed when j is 0. One way to correct this error is to have the loop terminate before j equals 0, which can be done by changing the condition j >= 0 to j > 0.

FINAL Unit2 MCQ - Topic order

94. Consider the following code segment.

```
for (int m = 16; m > 0; m -= 2)
{
  if ((m % 3) == 1)
  {
    System.out.print(m + " ");
  }
}
```

What is printed as a result of executing this code segment?

(A)

15 12 9 6 3

(B)

12 6 0

(C)

16 14 12 10 8 6 4 2

(D) ✓

16 10 4

Answer D

Correct. The code segment initializes m to 16 and decrements m by 2 in each iteration, and prints m only when m % 3 == 1. The loop exits when m is no longer greater than 0.

FINAL Unit2 MCQ - Topic order

95. Consider the following code segment.

```
int sum = 0;
for (int j = 1; /* missing condition */; j += 2)
{
  sum += j;
}
```

Which of the following replacements for /* missing condition */ will cause an infinite loop?

(A)

```
j == 100
```

(B) ✓

```
j != 100
```

(C)

```
j <= 100
```

(D)

```
j >= 100
```

Answer B

Correct. Using this expression, the loop will start with j equal to 1 and will repeatedly increment j by 2. Eventually, the value of j will go from 99 to 101 and never be exactly equal to 100. The loop condition will always be true, leading to an infinite loop.

FINAL Unit2 MCQ - Topic order

96. Consider the following code segment.

```
for (int k = 0; k < 20; k = k + 2)
{
  if (k % 3 == 1)
  {
    System.out.print(k + "  ");
  }
}
```

What is printed as a result of executing the code segment?

(A) 4 10 16 ✓

(B) 0 6 12 18

(C) 1 4 7 10 13 16 19

(D) 0 2 4 6 8 10 12 14 16 18

97. Consider the following code segment.

```
int sum = 0;
for (int k = 1; k <= 30; k = k + 2)
{
  sum += k;
}
System.out.println(sum);
```

Which of the following best describes the output of the code segment?

(A) The segment will print the sum of only the even integers from 1 through 30, inclusive, because it starts `sum` at `0`, increments `k` by `2`, and terminates when `k` exceeds `30`.

(B) The segment will print the sum of only the odd integers from 1 through 30, inclusive, because it starts `k` at `1`, increments `k` by `2`, and terminates when `k` exceeds `30`. ✓

(C) The segment will print the sum of only the even integers from 1 through 60, inclusive, because it starts `sum` at `0`, increments `k` by `2`, and iterates 30 times.

(D) The segment will print the sum of only the odd integers from 1 through 60, inclusive, because it starts `k` at `1`, increments `k` by `2`, and iterates 30 times.

Answer B

Correct. The code segment prints the sum of only the odd integers in the range from 1 to 30, inclusive.

FINAL Unit2 MCQ - Topic order

98. Consider the following code segment.

```
int num = 1;
for (int k = 2; k < 10; k++)
{
 num = 0;
 num = num + k;
}
```

What will be the value of `num` after the loop is executed?

(A) `9` ✓

(B) `10`

(C) `44`

(D) `45`

Answer A

Correct. Since the first statement in the loop body sets `num` to `0`, the second statement in the loop body has the effect of setting `num` to `0` plus the current value of `k`. In the last iteration of the loop, `k` has the value `9` and `num` is set to `9`.

FINAL Unit2 MCQ - Topic order

99. Consider the following code segment.

```
for (int j = 0; j < 4; j++)
{
  for (/* missing code */)
  {
    System.out.print(j + " ");
  }
  System.out.println();
}
```

This code segment is intended to produce the following output.

```
0
1 1
2 2 2
3 3 3 3
```

Which of the following can be used to replace /* missing code */ so that this code segment works as intended?

(A)

```
int k = 0; k < j; k++
```

(B) ✓

```
int k = 0; k <= j; k++
```

(C)

```
int k = j; k < 4; k++
```

(D)

```
int k = j; k <= 4; k++
```

Answer B

Correct. The outer loop counts from j = 0 up to j = 3, incrementing by 1. To produce the given output, the inner loop should count up from k = 0 to k = j. This will cause one value to be printed in the first iteration of the outer

FINAL Unit2 MCQ - Topic order

loop, two values to be printed in the second iteration of the outer loop, and so on.

FINAL Unit2 MCQ - Topic order

100. Consider the following code segment.

```
for (int j = 4; j > 0; j--)
{
  for (/* missing code */)
  {
    System.out.print(k + " ");
  }
  System.out.println();
}
```

This code segment is intended to produce the following output.

```
0 1 2 3
0 1 2
0 1
0
```

Which of the following can be used to replace /* missing code */ so that this code segment works as intended?

(A) ✓

```
int k = 0; k < j; k++
```

(B)

```
int k = 0; k <= j; k++
```

(C)

```
int k = j; k > 0; k--
```

(D)

```
int k = j; k >= 0; k--
```

Answer A

Correct. The outer loop counts from j = 4 down to j = 1, decrementing by 1. To produce the given output, the

inner loop should count up from k = 0 to k = j.

101. Consider the following method.

```
/** Precondition: num > 0 */
public static int doWhat(int num)
{
  int var = 0;
  for (int loop = 1; loop <= num; loop = loop + 2)
  {
    var += loop;
  }
  return var;
}
```

Which of the following best describes the value, if any, returned from a call to doWhat?

(A) The sum of all integers between 1 and num, inclusive

(B) The sum of all even integers between 1 and num, inclusive

(C) The sum of all odd integers between 1 and num, inclusive ✓

(D) No value is returned because of an infinite loop.

Answer C

Correct. The variable loop starts at 1 and increments by 2 for each iteration until it is greater than num. The values of loop that are being added together are the odd integers from 1 to num if num is odd or 1 to num - 1 if num is even. In either case, these are the odd integers between 1 and num.

FINAL Unit2 MCQ - Topic order

102. In the following code segment, num has been assigned a positive int value.

```
int x = 0;
int temp = num;

while (temp > 0)
{
  x += temp % 10;
  temp /= 10;
}

System.out.println(x);
```

Which of the following best describes the value printed by this code segment?

(A) A count of the number of digits in num

(B) The average of the digits in num

(C) The sum of the digits in num ✓

(D) Whether num is evenly divisible by 10

Answer C

Correct. The code segment uses modulus to separate the digits and adds them within a loop.

FINAL Unit2 MCQ - Topic order

103. Consider the following code segment.

```
int a = 24;
int b = 30;
while (b != 0)
{
  int r = a % b;
  a = b;
  b = r;
}

System.out.println(a);
```

What is printed as a result of executing the code segment?

(A) 0

(B) 6 ✓

(C) 24

(D) 30

FINAL Unit2 MCQ - Topic order

104. In the following code segment, n is a properly declared and initialized int variable whose value is positive.

```
while (n > 0)          // Line 1
{                      // Line 2
  n /= 10;             // Line 3
}                      // Line 4
System.out.println(n);  // Line 5
```

This code segment is intended to print the leftmost digit of n. For example, if n is 302, this code segment should print 3. However, this code segment is not working as intended. Which of the following changes can be made so that this code segment works as intended?

(A) In line 1, changing the condition n > 0 to n / 10 > 0 ✓

(B) In line 1, changing the condition n > 0 to n % 10 > 0

(C) In line 3, changing the statement n /= 10 to n -= 10

(D) In line 3, changing the statement n /= 10 to n %= 10

Answer A

Correct. The given code segment will print 0 for all positive values of n. The while loop repeatedly divides n by 10, which removes the rightmost digit of n until n is 0. The loop then terminates, printing 0.

To correct this error, the loop should terminate when the leftmost digit is reached, which occurs when n / 10 is 0. For example, if n is initially 302, the first iteration of the loop will change n to 30 and the second iteration of the loop will change n to 3. The loop will terminate at this point because 3 / 10 is 0.

FINAL Unit2 MCQ - Topic order

105. In the following code segment, the variable `num` is assigned a random value inside a loop.

```
int result = 0;
for (int j = 1; j <= 20; j++)
{
   int num = ((int) Math.random() * 10) + 1;
   if (num > result)
   {
      result = num;
   }
}
```

Which of the following best describes the value of the variable `result` after executing the code segment?

(A) The first random value assigned to `num`

(B) The last random value assigned to `num`

(C) The least random value assigned to `num`

(D) The greatest random value assigned to `num` ✓

Answer D

Correct. Each time a random value is assigned to `num`, that value is compared to `result`. If `num` is greater than `result`, then `result` is updated to the value of `num`. After the loop finishes, `result` will have the greatest value of the random numbers that were generated.

FINAL Unit2 MCQ - Topic order

106. In the following code segment, assume that low and high are properly declared and initialized int variables and that low < high. The code segment is intended to print the sum of the integers between low and high, inclusive, but does not always work as intended.

```
int sum = 0;        // line 1
int j = low;        // line 2
while (sum <= high)  // line 3
{
  sum += j;         // line 5
  j++;              // line 6
}
System.out.println(sum);
```

Which of the following changes can be made so that this code segment works as intended?

(A) Changing line 1 to int sum = low;

(B) Changing line 2 to int j = 0;

(C) Changing line 3 to while (j <= high) ✓

(D) Interchanging lines 5 and 6

Answer C

Correct. In the given code segment, the loop terminates when sum exceeds high. In order for the code segment to add all the integers from low to high, the loop should continue until j exceeds high.

FINAL Unit2 MCQ - Topic order

107. Consider the following code segment.

```
int num = 245;
int temp = num;

while (temp > 0)
{
  System.out.print(temp % 10 + " ");
  temp /= 10;
}
```

What is printed as a result of executing this code segment?

(A)

2 4 5

(B)

4 5

(C)

5 4

(D) ✓

5 4 2

Answer D

Correct. In the while loop, this code segment uses the expression temp % 10 to print the rightmost digit of temp, then discards the rightmost digit by dividing temp by 10 using integer division. Repeating this process will print the digits of num from right to left.

FINAL Unit2 MCQ - Topic order

108. Consider the following code segment.

```
int x = 1;
while (x < 10)
{
  if (x % 2 == 0)
  {
    System.out.print(x + " ");
  }
  /* missing code */
}
```

Which of the following replacements for /* missing code */ will cause the code segment to print nothing?

(A)

```
x++;
```

(B) ✓

```
x += 2;
```

(C)

```
x += 3;
```

(D)

```
x *= 2;
```

Answer B

Correct. The loop will execute for x having the values 1, 3, 5, 7, and 9. When x becomes 11 the loop will terminate. Even though the loop executes multiple times, the values assigned to x are not even, so nothing is printed.

FINAL Unit2 MCQ - Topic order

109. Consider the following code segment.

```
int k = 1;
while (k < 20)
{
  if ((k % 3) == 1)
    System.out.print(k + " ");

  k++;
}
```

What is printed as a result of executing this code segment?

(A) `2 5 8 11 14 17`

(B) `3 6 9 12 15 18`

(C) `1 4 7 10 13 16 19` ✓

(D) `1 3 5 7 9 11 13 15 17 19`

FINAL Unit2 MCQ - Topic order

110. An acronym is formed by extracting the first letter of each word in a phrase of one or more words. For example, the acronym of the phrase “Laugh Out Loud” is “LOL”, and the acronym of the phrase “If you know you know” is “Iykyk.”

A phrase must meet the following conditions.

- The phrase has exactly one space between each word.
- The phrase does not have leading or trailing spaces.
- The final word in the phrase has at least two characters.

In the following code segment, the String variable phrase has been properly declared and initialized such that it meets these conditions. The code segment is intended to create the acronym of the String stored in phrase and store it in the variable text.

```
String temp = phrase;
String text = temp.substring(0, 1);
int i = temp.indexOf(" ");
while (i > 0)
{
  /* missing code */
}
```

Which of the following can be used to replace /* missing code */ so that the code segment works as intended?

FINAL Unit2 MCQ - Topic order

(A)

```
text += temp.substring(0, 1);

temp = temp.substring(i + 1);

i = temp.indexOf(" ");
```

(B) ✓

```
temp = temp.substring(i + 1);

text += temp.substring(0, 1);

i = temp.indexOf(" ");
```

(C)

```
text += temp.substring(i, i + 1);

i = i + 1;
```

(D)

```
text += temp.substring(i, i + 1);

i = temp.indexOf(" ");
```

Answer B

Correct. The given code segment initially assigns the value of phrase to temp. The given code segment extracts the first character from the first word in temp and assigns it to text before the while loop. Using this replacement for /* missing code */ inside the loop will correctly extract the first character of the second and subsequent words from temp and concatenate them to text by repeatedly removing the first word from temp, finding the next space, and concatenating the next word's first character to text.

FINAL Unit2 MCQ - Topic order

111. Consider the following method.

```
public static String changeStr(String str)
{
  String result = "";
  for (int i = str.length() - 1; i >= str.length() / 2; i -= 2)
  {
    result += str.substring(i, i + 1);
  }
  return result;
}
```

What value is returned as a result of the method call `changeStr("12345")`?

(A)

"4"

(B) ✓

"53"

(C)

"531"

(D)

"543"

Answer B

Correct. The `for` loop assigns the values `4` and then `2` to the loop control variable `i`. When `i` has the value `4`, the single-character substring of `str` containing `"5"` is assigned to result. When `i` has the value `2`, the single-character substring containing `"3"` is concatenated with `"5"` to produce the string `"53"`.

FINAL Unit2 MCQ - Topic order

112. In the following code segment, str is a properly declared and initialized string. The code segment is intended to count the number of times that "a" appears in str.

```
int count = 0;
String temp = str;
int loc = temp.indexOf("a");

while (loc >= 0)
{
  /* missing code */
}
```

Which of the following can be used to replace /* missing code */ so that the code segment works as intended?

(A)

```
count++;

temp = str.substring(loc + 1);
```

(B)

```
count++;

loc++;

temp = str.substring(loc);
```

(C)

```
count++;

temp = temp.substring(loc);

loc = temp.indexOf("a");
```

(D) ✓

```
count++;

temp = temp.substring(loc + 1);

loc = temp.indexOf("a");
```

FINAL Unit2 MCQ - Topic order

Answer D

Correct. Using this code segment, after each "a" is found, count is incremented by 1, temp is assigned the substring starting immediately after the "a" that was found, and loc is assigned the index of the next "a". When there are no more "a"s remaining, indexOf will return -1, causing the loop to terminate.

FINAL Unit2 MCQ - Topic order

113. Consider the following code segment.

```
String str = "ABCDEF";
String result = "";
/* missing code */
System.out.println(result);
```

Which of the following can be used to replace `/* missing code */` so that this code segment prints the string `"EFCDAB"`?

FINAL Unit2 MCQ - Topic order

(A)

```
for (int j = str.length() - 1; j >= 0; j -= 1)
{
  result += str.substring(j, j + 1);
}
```

(B)

```
for (int j = str.length() - 1; j >= 0; j -= 2)
{
  result += str.substring(j, j + 2);
}
```

(C)

```
for (int j = str.length() - 2; j >= 0; j -= 1)
{
  result += str.substring(j, j + 2);
}
```

(D) ✓

```
for (int j = str.length() - 2; j >= 0; j -= 2)
{
  result += str.substring(j, j + 2);
}
```

Answer D

Correct. In the first iteration of the loop, j = 4, causing the loop body to append the substring of str from index 4 to 5 (or "EF") to result. In the second iteration of the loop, j = 2, causing the loop body to append the substring

of str from index 2 to 3 (or "CD") to result. In the third and final iteration of the loop, j = 0, causing the loop body to append the substring of str from index 0 to 1 (or "AB") to result.

114. Consider the following method.

```
public static String mystery(String input)
{
  String output = "";
  for (int k = 1; k < input.length(); k = k + 2)
  {
    output += input.substring(k, k + 1);
  }
  return output;
}
```

What is returned as a result of the call mystery("computer")?

(A)

"computer"

(B)

"cmue"

(C) ✓

"optr"

(D)

"ompute"

Answer C

Correct. The value of k starts at 1 and is incremented by 2 as long as it is less than input.length(). In this case, input is assigned "computer", so input.length() is 8. The values of k are 1, 3, 5, 7, and then when k is 9, the loop terminates. The call input.substring(k, k + 1) will return the value of input at index k. The values added to output are, in order, "o", "p", "t", and "r".

FINAL Unit2 MCQ - Topic order

115. Consider the following code segment.

```
String str = "qrstu";
String result = "";
for (int j = 0; j < str.length(); j++)
{
  result += str.substring(0, j + 1);
}
System.out.println(result);
```

What, if anything, is printed as a result of executing this code segment?

(A)

rrsrstrstu

(B)

qqrqrsqrst

(C) ✓

qqrqrsqrstqrstu

(D) Nothing is printed because an IndexOutOfBoundsException occurs.

Answer C

Correct. In the first iteration of the loop, j = 0, causing the loop body to append the substring of str from index 0 to 0 (or "q") to result. In the second iteration of the loop, j = 1, causing the loop body to append the substring of str from index 0 to 1 (or "qr") to result. In the third iteration of the loop, j = 2, causing the loop body to append the substring of str from index 0 to 2 (or "qrs") to result. In the fourth iteration of the loop, j = 3, causing the loop body to append the substring of str from index 0 to 3 (or "qrst") to result. In the fifth and last iteration of the loop, j = 4, causing the loop body to append the substring of str from index 0 to 4 (or "qrstu") to result.

FINAL Unit2 MCQ - Topic order

116. Consider the following code segment.

```
int j = 1;
while (j < 5)
{
  int k = 1;
  while (k < 5)
  {
    System.out.println(k);
    k++;
  }
  j++;
}
```

Which of the following best explains the effect, if any, of changing the first line of code to `int j = 0;` ?

(A) There will be one more value printed because the outer loop will iterate one additional time.

(B) There will be four more values printed because the outer loop will iterate one additional time. ✓

(C) There will be one less value printed because the outer loop will iterate one fewer time.

(D) There will be no change to the output of the code segment.

Answer B

Correct. In the given code segment, the outer loop iterates four times (from when `j` is `1` to when `j` is `4`). The inner loop also iterates four times (from when `k` is `1` to when `k` is `4`). A total of sixteen values are printed. By changing the first line of code to `j = 0;`, the outer loop will iterate one additional time, causing the inner loop to iterate four additional times. Therefore, four additional values will be printed.

FINAL Unit2 MCQ - Topic order

117. Consider the following code segment.

```
for (int x = 0; x <= 4; x++)      // Line 1
{
 for (int y = 0; y < 4; y++)    // Line 3
 {
   System.out.print("a");
 }
 System.out.println();
}
```

Which of the following best explains the effect of simultaneously changing `x <= 4` to `x < 4` in line 1 and `y < 4` to `y <= 4` in line 3 ?

(A) `"a"` will be printed fewer times because while each output line will have the same length as before, the number of lines printed will decrease by 1.

(B) `"a"` will be printed more times because while the number of output lines will be the same as before, the length of each output line will increase by 1.

(C) `"a"` will be printed the same number of times because while the number of output lines will decrease by 1, the length of each line will increase by 1. ✓

(D) The output of the code segment will not change in any way.

Answer C

Correct. The original code prints 5 lines of output with 4 characters on each line, resulting in a total of 20 characters; the new code will print 4 lines of output with 5 characters on each line, resulting in the same total of 20 characters.

FINAL Unit2 MCQ - Topic order

118. Consider the following code segment.

```
for (int outer = 0; outer < 3; outer++)
{
  for ( /* missing loop header */ )
  {
    System.out.print(outer + "" + inner + "_");
  }
}
```

Which of the following can be used as a replacement for `/* missing loop header */` so that the code segment produces the output `00_01_02_11_12_22_`?

(A)

```
int inner = 0; inner < 3; inner++
```

(B)

```
int inner = 1; inner < 3; inner++
```

(C)

```
int inner = outer - 1; inner < 3; inner++
```

(D) ✓

```
int inner = outer; inner < 3; inner++
```

Answer D

Correct. When `outer` has the value `0`, `inner` is assigned values from `0` to `2`. When `outer` has the value `1`, `inner` is assigned values from `1` to `2`. When `outer` has the value `2`, `inner` is assigned only the value `2`.

FINAL Unit2 MCQ - Topic order

119. Consider the following output.

```
1   1   1   1   1
2   2   2   2
3   3   3
4   4
5
```

Which of the following code segments will produce this output?

(A)
```
for (int j = 1; j <= 5; j++)
{
  for (int k = 1; k <= j; k++)
  {
    System.out.print(j + "  ");
  }
  System.out.println();
}
```

(B)
```
for (int j = 1; j <= 5; j++)
{
  for (int k = 5; k >= 1; k--)
  {
    System.out.print(j + "  ");
  }
  System.out.println();
}
```

(C) ✓
```
for (int j = 1; j <= 5; j++)
{
  for (int k = 5; k >= j; k--)
  {
    System.out.print(j + "  ");
  }
  System.out.println();
}
```

(D)
```
for (int j = 1; j <= 5; j++)
{
  for (int k = j; k <= 5; k++)
  {
    System.out.print(k + "  ");
  }
  System.out.println();
}
```

FINAL Unit2 MCQ - Topic order

120. Consider the following code segment.

```
int count = 0;
for (int x = 1; x <= 3; x++)
{
  /* missing loop header */
  {
    count++;
  }
}
System.out.println(count);
```

Which of the following can be used to replace `/* missing loop header */` so that the code segment prints `6`?

(A)

```
for (int y = 0; y <= 2; y++)
```

(B)

```
for (int y = 0; y < 3; y++)
```

(C)

```
for (int y = 3; y <= x; y++)
```

(D) ✓

```
for (int y = 0; y < x; y++)
```

Answer D

Correct. In the first iteration of the outer loop, the inner loop iterates one time. In the second iteration of the outer loop, the inner loop iterates two times. In the third (and last) iteration of the outer loop, the inner loop iterates three times. Since the body of the inner loop executes a total of six times, the code segment prints `6`.

FINAL Unit2 MCQ - Topic order

121. Consider the following code segment.

```
int a = 1;
while (a <= 2)
{
  int c = 1;
  while (/* missing condition */)
  {
    System.out.print("*");
    c++;
  }
  a++;
}
```

The code segment is intended to print "******".

Which of the following can be used to replace /* missing condition */ so that the code segment works as intended?

(A)

c > 2

(B)

c < 3

(C) ✓

c <= 3

(D)

c >= 3

Answer C

Correct. The outer loop iterates two times (when a is 1 and when a is 2). For six asterisks to be printed, the inner loop needs to iterate three times for each iteration of the outer loop. If the condition of the inner loop is c <= 3, the inner loop will iterate three times.

FINAL Unit2 MCQ - Topic order

122. Consider the following code segment.

```
for (int outer = 1; outer <= 4; outer++)
{
  for (int inner = outer; inner <= 4; inner++)
  {
    System.out.print(inner + " ");
  }
  System.out.println();
}
```

What is printed as a result of executing the code segment?

FINAL Unit2 MCQ - Topic order

(A)

1 1 1 1

2 2 2

3 3

4

(B)

1 1 1 1

2 2 2 2

3 3 3 3

4 4 4 4

(C) ✓

1 2 3 4

2 3 4

3 4

4

(D)

1 2 3 4

1 2 3 4

1 2 3 4

1 2 3 4

Answer C

Correct. The outer loop iterates four times for when outer is assigned the values 1 through 4. For each iteration, the number of times the inner loop iterates is dependent on the value of outer. When outer is 1, the inner loop

FINAL Unit2 MCQ - Topic order

iterates from 1 to 4. When outer is 2, the inner loop iterates from 2 to 4. When outer is 3, the inner loop iterates from 3 to 4. When outer is 4, the inner loop iterates one time and prints 4 followed by a space.

FINAL Unit2 MCQ - Topic order

123. Consider the following code segment.

```
/* missing loop header */
{
  for (int k = 0; k < 4; k++)
  {
    System.out.print(k);
  }
  System.out.println();
}
```

This code segment is intended to produce the following output.

```
0123
0123
0123
```

Which of the following can be used to replace /* missing loop header */ so that the code segment works as intended?

(A)

```
for (int j = 0; j < 2; j++)
```

(B)

```
for (int j = 1; j <= 2; j++)
```

(C)

```
for (int j = 0; j <= 6; j += 2)
```

(D) ✓

```
for (int j = 1; j < 6; j += 2)
```

Answer D

Correct. To produce the desired output, the outer `for` loop must iterate three times. This `for` loop iterates three times, when `j` is `1`, `3`, and `5`.

FINAL Unit2 MCQ - Topic order

124. Consider the following output.

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
1 2 3 4 5 6
```

Which of the following code segments will produce the output shown above?

(A)
```
for (int j = 1; j <= 6; j++)
{
  for (int k = 1; k < j; k++)
    System.out.print(" " + k);
  System.out.println();
}
```

(B)
```
for (int j = 1; j <= 6; j++)
{
  for (int k = 1; k <= j; k++)
    System.out.print(" " + j);
  System.out.println();
}
```

(C) ✓
```
for (int j = 1; j <= 6; j++)
{
  for (int k = 1; k <= j; k++)
    System.out.print(" " + k);
  System.out.println();
}
```

(D)
```
for (int j = 1; j < 6; j++)
{
  for (int k = 1; k <= j; k++)
    System.out.print(" " + k);
  System.out.println();
}
```

FINAL Unit2 MCQ - Topic order

125. Consider the following code segment.

```
for (int j = 0; j < 3; j++)
{
 for (int k = 0; k < 4; k++)
 {
   System.out.println("Fun");
 }
}
```

Which of the following best explains how changing the outer `for` loop header to `for (int j = 0; j <= 3; j++)` affects the output of the code segment?

(A) The output of the code segment will be unchanged.

(B) The string `"Fun"` will be printed more times because the outer loop will execute more times. ✓

(C) The string `"Fun"` will be printed more times because the inner loop will execute more times in each iteration of the outer loop.

(D) The string `"Fun"` will be printed fewer times because the outer loop will execute fewer times.

Answer B

Correct. Changing `j < 3` to `j <= 3` will cause the outer loop to iterate one additional time. The original code segment prints `"Fun"` 12 times while the revised code segment prints `"Fun"` 16 times.

FINAL Unit2 MCQ - Topic order

126. Consider the following code segment.

```
for (int j = 1; j <= 4; j++)        // Line 1
{
  for (int k = j + 1; k >= 1; k--)  // Line 3
  {
    System.out.print(j + " ");      // Line 5
  }
  System.out.println();
}
```

The code segment is intended to produce the following output, but does not work as intended.

```
1
2 2
3 3 3
4 4 4 4
```

Which of the following changes can be made so that the code segment works as intended?

(A) Changing line 1 to `for (int j = 1; j < 4; j++)`

(B) Changing line 3 to `for (int k = 1; k <= j + 1; k++)`

(C) Changing line 3 to `for (int k = j; k >= 1; k--)` ✓

(D) Changing line 5 to `System.out.print(k + " ");`

Answer C

Correct. The given code segment produces the following output because the inner loop iterates one too many times for each iteration of the outer loop.

```
1 1
2 2 2
3 3 3 3
4 4 4 4 4
```

FINAL Unit2 MCQ - Topic order

Having the inner loop start at `k = j` instead of `k = j + 1` will cause the inner loop to iterate the intended number of times.

127. Consider the following code segment.

```
for (int outer = 0; outer < n; outer++)
{
  for (int inner = 0; inner <= outer; inner++)
  {
    System.out.print(outer + " ");
  }
}
```

If `n` has been declared as an integer with the value 4, what is printed as a result of executing the code segment?

(A) `0 1 2 3`

(B) `0 1 2 2 3 3 3`

(C) `0 1 1 2 2 2 3 3 3 3` ✓

(D) `0 0 1 0 1 2 0 1 2 3`

FINAL Unit2 MCQ - Topic order

128. Consider the following method.

```
public int sol(int lim)
{
  int s = 0;

  for (int outer = 1; outer <= lim; outer++)
  {
    for (int inner = outer; inner <= lim; inner++)
    {
      s++;
    }
  }

  return s;
}
```

What value is returned as a result of the call `sol(10)`?

(A) 20

(B) 45

(C) 55 ✓

(D) 100

FINAL Unit2 MCQ - Topic order

129. The following code segment appears in a method in the same class as the method doSomething().

```
int i = 1;
while (i < 3)
{
   for (int j = 1; j <= 5; j++)
   {
      doSomething();
   }
   i++;
}
```

How many times will doSomething() be called as a result of executing this code segment?

(A) 5

(B) 8

(C) 10 ✓

(D) 15

Answer C

Correct. The outer loop runs 2 times. For each iteration of the outer loop, the inner loop runs 5 times. Therefore, doSomething() will be called 2 × 5 times.

FINAL Unit2 MCQ - Topic order

130. Consider the following code segment.

```
for (int num = 0; num < 10; num += 2)
{
 for (int val = 0; val < 5; val++)
 {
   System.out.println("hop");
 }
}
```

How many times will `System.out.println("hop")` be executed?

(A) 5

(B) 10

(C) 25 ✓

(D) 50

Answer C

Correct. The body of the outer `for` loop executes 5 times as the variable `num` takes on the values `0`, `2`, `4`, `6`, and `8`. For each value of `num`, the body of the inner `for` loop executes 5 times as the variable `val` takes on the values `0`, `1`, `2`, `3`, and `4`. The value of `hop` is printed a total of $5 \times 5 = 25$ times.

FINAL Unit2 MCQ - Topic order

131. Consider the following code segment.

```
int j = 1;
while (j < 4)
{
  for (int k = 0; k <= 4; k++)
  {
    System.out.println("hello");     // line 6
  }
  j++;
}
```

How many times will the statement in line 6 be executed as a result of running this code segment?

(A) 12

(B) 15 ✓

(C) 16

(D) 20

Answer B

Correct. The outer loop iterates 3 times (starting at j = 1 and terminating when j = 4). The inner loop iterates 5 times (starting at k = 0 and terminating when k exceeds 4). The print statement occurs in the inner loop, so it is executed 3 * 5 = 15 times.

FINAL Unit2 MCQ - Topic order

132. Consider the following code segment.

```
for (int outer = 0; outer < 5; outer++)
{
  for (int inner = 4; inner >= 0; inner--)
  {
    System.out.println("*");              // Line 5
  }
}
```

How many times will the statement in line 5 be executed as a result of running this code segment?

(A) 20

(B) 24

(C) 25 ✓

(D) 30

Answer C

Correct. The outer loop iterates 5 times (going from outer = 0 to outer = 4). The inner loop iterates 5 times (going from inner = 4 to inner = 0). The print statement occurs in the inner loop, so it is executed 5 × 5 = 25 times.

FINAL Unit2 MCQ - Topic order

133. Consider the following code segment.

```
for (int x = 1; x <= 3; x++)
{
   for (int y = 5; y > 1; y--)
   {
      System.out.print("*");
   }
}
```

Which of the following best describes the behavior of this code segment?

(A) The print statement is executed 7 times because the outer loop iterates 3 times and the inner loop iterates 4 times.

(B) The print statement is executed 8 times because the outer loop iterates 3 times and the inner loop iterates 5 times.

(C) The print statement is executed 12 times because the outer loop iterates 3 times and the inner loop iterates 4 times for each iteration of the outer loop. ✓

(D) The print statement is executed 15 times because the outer loop iterates 3 times and the inner loop iterates 5 times for each iteration of the outer loop.

Answer C

Correct. The outer loop runs 3 times (when x is 1, 2, and 3). For each iteration of the outer loop, the inner loop runs 4 times (when y is 5, 4, 3, and 2).

FINAL Unit2 MCQ - Topic order

134. Consider the following code segment.

```
int counter = 0;
for (int x = 10; x > 0; x--)
{
  for (int y = x; y <= x; y++)
  {
    counter++;       // Line 6
  }
}
```

How many times will the statement in line 6 be executed as a result of running the code segment?

(A) 1

(B) 10 ✓

(C) 11

(D) 20

Answer B

Correct. In the inner loop, the initial condition is `y = x`, which means that the boundary condition, `y <= x`, will immediately be met and the inner loop will only execute one time. So `counter` will be incremented for each iteration of the outer loop (i.e., 10 times).

FINAL Unit2 MCQ - Topic order

135. Consider the following code segment.

```
double sum = 0.0;
for (int outer = 1; outer <= 4; outer++)
{
   for (int inner = 0; inner <= 4; inner++)
   {
      sum += Math.random();
   }
}
```

How many times will Math.random() be called as a result of executing this code segment?

(A) 5

(B) 9

(C) 16

(D) 20 ✓

Answer D

Correct. The outer loop runs 4 times. For each iteration of the outer loop, the inner loop runs 5 times. Therefore, the Math.random() method is called 4 × 5 times.

FINAL Unit2 MCQ - Topic order

136. Consider the following code segment.

```
int val = 1;
while (val <= 6)
{
  for (int k = 0; k <= 2; k++)
  {
    System.out.println("Surprise!");   // Line 6
  }
  val++;
}
```

How many times will the statement in line 6 be executed as a result of running the code segment?

(A) 3

(B) 12

(C) 15

(D) 18 ✓

Answer D

Correct. The `while` loop executes 6 times. During each iteration of the `while` loop, the `for` loop iterates 3 times (when `k` is `0`, `1`, and `2`) and prints the string `"Surprise!"` each time, for a total of 18 times (6×3).

FINAL Unit2 MCQ - Topic order

137. In the following code segment, `outerMax` and `innerMax` are properly declared and initialized `int` variables that are greater than or equal to zero.

```
for (int outer = 0; outer < outerMax; outer++)
{
  for (int inner = 0; inner < innerMax; inner++)
  {
    System.out.println("*");
  }
}
```

Which of the following best describes the behavior of the code segment?

(A) It prints `"*"` a total of `outerMax + innerMax` times.

(B) It prints `"*"` a total of `outerMax * innerMax` times. ✓

(C) It prints `"*"` a total of `(outerMax + 1) + (innerMax + 1)` times.

(D) It prints `"*"` a total of `(outerMax + 1) * (innerMax + 1)` times.

Answer B

Correct. The outer `for` loop iterates `outerMax` times. For each iteration of the outer `for` loop, the inner `for` loop iterates `innerMax` times. Therefore, the number of times `"*"` is printed is `outerMax * innerMax`.

FINAL Unit2 MCQ - Topic order

138. Consider the following two code segments. Assume that the int variables m and n have been properly declared and initialized and are both greater than 0.

Code Segment 1

```
for (int i = 0; i < m * n; i++)
{
  System.out.print("*");
}
```

Code Segment 2

```
for (int j = 1; j <= m; j++)
{
  for (int k = 1; k < n; k++)
  {
    System.out.print("@");
  }
}
```

Assume that the initial values of m and n are the same in Code Segment 1 as they are in Code Segment 2. Which of the following correctly compares the number of times that "*" and "@" are printed when each code segment is executed?

(A) "*" is printed m fewer times than "@".

(B) "*" is printed n fewer times than "@".

(C) "*" is printed m more times than "@". ✓

(D) "*" is printed n more times than "@".

Answer C

Correct. There are m * n iterations of the loop in Code Segment 1. In Code Segment 2, the outer loop executes m times and the inner loop executes n - 1 times for each iteration of the outer loop. There are m * n - m iterations of the inner loop in code segment 2, so "*" is printed m more times than "@" is printed.

FINAL Unit2 MCQ - Topic order

139. The question refer to the following code segment.

```
int k = a random number such that 1 ≤ k ≤ n ;
for (int p = 2; p <= k; p++)
  for (int r = 1; r < k; r++)
    System.out.println("Hello");
```

What is the maximum number of times that `Hello` will be printed?

(A) $n - 1$

(B) $n - 2$

(C) $(n - 1)^2$ ✓

(D) n^2

140. The question refer to the following code segment.

```
int k = a random number such that 1 ≤ k ≤ n ;
for (int p = 2; p <= k; p++)
  for (int r = 1; r < k; r++)
    System.out.println("Hello");
```

What is the minimum number of times that `Hello` will be printed?

(A) 0 ✓

(B) 1

(C) 2

(D) $n - 1$

FINAL Unit2 MCQ - Topic order

141. Consider the following code segment.

```
int value = 100;
while (value % 5 == 0)
{
  value /= 2;
}
System.out.println(value);
```

What is printed as a result of executing this code segment?

(A) ✓

12

(B)

13

(C)

25

(D)

100

Answer A

Correct. The code segment repeatedly divides value by 2 until it is no longer a multiple of 5. It is initially 100, then 50, then 25, then 12. The last value is 12 instead of 12.5 because when dividing int values, the result is only the integer portion of the quotient. Since 12 is not a multiple of 5, the loop terminates and this value is printed.

FINAL Unit2 MCQ - Topic order

142. Consider the following expression.

(3 + 4 == 5) != (3 + 4 >= 5)

What value, if any, does the expression evaluate to?

(A) ✓
true

(B)
false

(C)
5

(D) No value; relational operators cannot be used on arithmetic expressions.

Answer A

Correct. The expression 3 + 4 == 5 evaluates to false. The expression 3 + 4 >= 5 evaluates to true. These two expressions differ in value, so (3 + 4 == 5) != (3 + 4 >= 5) evaluates to true.

FINAL Unit2 MCQ - Topic order

143. Consider the following code segment.

```
int quant = 20;
int unitPrice = 4;
int ship = 8;
int total;
if (quant > 10)
{
  unitPrice = 3;
}
if (quant > 20)
{
  ship = 0;
}
total = quant * unitPrice + ship;
```

What is the value of total after this code segment is executed?

(A)

20

(B)

60

(C) ✓

68

(D)

88

Answer C

Correct. The value of unitPrice is initially set to 4 and then it is changed to 3 because quant > 10 is true. The value of ship is initially set to 8 and then it is not changed because quant > 20 is false. The value of total is 20 * 3 + 8, which is equal to 68.

FINAL Unit2 MCQ - Topic order

144. Assume that `x` and `y` are variables of type `int`. Which of the following Java expressions never results in a division by zero?

(A) `((y / x) == 0) && (x != 0)`

(B) `((y / x) == 0) || (x != 0)`

(C) `(x != 0) && ((y / x) == 0)` ✓

(D) `(x != 0) || ((y / x) == 0)`

Answer C

Correct. The operands of the `&&` operator are evaluated in left-to-right order. If the first operand evaluates to `false`, the result of the logical expression can be determined without evaluating the second operand, and so the second operand is not evaluated, in a process known as short-circuit evaluation. If `x` has the value `0`, the first operand of `&&` evaluates to `false` and the second operand, which would cause a division by zero, is not evaluated. The second operand is evaluated only when `x` has a value other than zero.

FINAL Unit2 MCQ - Topic order

145. Consider the following code segment.

```
boolean b1 = true && ((5 % 2) == 0);
boolean b2 = /* missing expression */ ;
```

Which of the following can replace `/* missing expression */` so that the value stored in `b2` is the same as the value stored in `b1` after the code segment has executed?

(A)

false || ((5 % 2) == 1)

(B) ✓

false && ((5 % 2) == 1)

(C)

true || ((5 % 2) == 1)

(D)

true && ((5 % 2) == 1)

Answer B

Correct. The expression `5 % 2 == 0` evaluates to `false`, and `true && false` evaluates to `false`, so `b1` is assigned the value `false`. The expression on the right-hand side of the assignment statement for `b2` evaluates to `false` by short-circuit evaluation.

FINAL Unit2 MCQ - Topic order

146. Consider the following methods.

```
/** Preconditions: a > 0 and b > 0 */
public static int methodX(int a, int b)
{
  int loopCount = 0;
  for (int i = 0; i < a / b; i++)
  {
    loopCount++;
  }
  return loopCount;
}
```

```
/** Preconditions: a > 0 and b > 0 */
public static int methodY(int a, int b)
{
  int loopCount = 0;
  int i = 0;
  while (i < a)
  {
    loopCount++;
    i += b;
  }
  return loopCount;
}
```

Which of the following best describes the conditions under which methodX and methodY return the same value?

(A) When a and b are both even

(B) When a and b are both odd

(C) When a % b is equal to zero ✓

(D) When a % b is equal to one

Answer C

Correct. The body of the for loop in methodX is executed a / b times. The body of the while loop in methodY is executed a / b times only when a % b is equal to 0. When a % b is not equal to 0, the body of the while loop in methodY is executed an additional time. For example, when a has the value 11 and b has the value 5, a / b evaluates to 2 and the for loop is executed two times but the while loop is executed three times.

FINAL Unit2 MCQ - Topic order

147. Consider the following code segment.

```
for (int k = 0; k < 9; k = k + 2)
{
 if ((k % 2) != 0)
 {
   System.out.print(k + " ");
 }
}
```

What, if anything, is printed as a result of executing the code segment?

(A) `0 2 4 6 8`

(B) `1 3 5 7 9`

(C) `1 3 5 7`

(D) Nothing is printed. ✓

Answer D

Correct. The loop control variable `k` takes on the values `0`, `2`, `4`, `6`, and `8` in the `for` loop. In the loop, values of `k` that are odd (not evenly divisible by two) are printed. Nothing is printed because `k` is always even.

FINAL Unit2 MCQ - Topic order

148. Consider the following code segment.

```
int num = 1;
int count = 0;
while (num <= 10)
{
  if (num % 2 == 0 && num % 3 == 0)
  {
    count++;
  }
  num++;
}
```

What value is stored in the variable count as a result of executing the code segment?

(A) ✓
1

(B)
3

(C)
5

(D)
7

Answer A

Correct. The code segment considers the integers from 1 to 10 and increments count for those integers that are divisible by both 2 and 3. There is only one integer that meets this criteria, which is 6.

FINAL Unit2 MCQ - Topic order

149. In the following code segment, num has been assigned a positive int value. The following code segment is intended to print true if num is even and is intended to print false otherwise.

```
boolean isEven = true;
if (/* missing code */)
{
  isEven = false;
}
System.out.println(isEven);
```

Which of the following can replace /* missing code */ so that this code segment works as intended?

(A)

num / 2 == 0

(B)

num / 2 != 0

(C)

num % 2 == 0

(D) ✓

num % 2 != 0

Answer D

Correct. This expression uses modulus to check whether an integer is odd. If so, the code segment sets isEven to false. Otherwise, the initial value of isEven, which is true, is unchanged.

FINAL Unit2 MCQ - Topic order

150. Consider the following code segment. Assume that the int variable input has been properly declared and initialized.

```
int answer = 1;
if (input != 0)
{
  int count = 1;

  while (count != input)
  {
    count++;
    answer *= count;
  }
}

System.out.println(answer);
```

Which of the following best describes the condition in which this code segment always results in integer overflow?

(A) When input == 0

(B) When input == 1

(C) When input > 0

(D) When input < 0 ✓

Answer D

Correct. Since count starts as 1 and increases with each iteration, count will never be equal to a negative number. The loop condition will remain true and count will overflow.

FINAL Unit2 MCQ - Topic order

151. Consider the following method.

```
public static int mystery(boolean a, boolean b, boolean c)
{
 int answer = 7;

 if (!a)
 {
   answer += 1;
 }

 if (b)
 {
   answer += 2;
 }

 if (c)
 {
   answer += 4;
 }

 return answer;
}
```

Which of the following method calls will return the value `11` ?

(A) `mystery(true, true, true)`

(B) `mystery(true, false, true)` ✓

(C) `mystery(false, true, false)`

(D) `mystery(false, false, true)`

Answer B

Correct. Since `a` is `true`, `!a` is `false` and the body of the first `if` statement is not executed. Since `b` is `false`, the body of the second `if` statement is not executed. Since `c` is `true`, the body of the third `if` statement is executed and the value `7 + 4 = 11` is returned.

FINAL Unit2 MCQ - Topic order

152. Consider the following code segment.

```
double regularPrice = 100.0;
boolean onClearance = true;
boolean hasCoupon = false;

double finalPrice = regularPrice;
if (onClearance)
{
  finalPrice -= finalPrice * 0.25;
}
if (hasCoupon)
{
  finalPrice -= 5.0;
}
System.out.println(finalPrice);
```

What is printed as a result of executing the code segment?

(A)

20.0

(B)

25.0

(C)

70.0

(D) ✓

75.0

Answer D

Correct. The variable finalPrice is initialized to regularPrice. Since the boolean variable onClearance is true, the code in the body of the first if statement is executed, setting finalPrice to 75.0. Since the boolean variable hasCoupon is false, the code in the body of the second if statement is not executed. The value 75.0 is printed.

FINAL Unit2 MCQ - Topic order

153. Consider the following class declaration

```
public class SomeClass
{
  private int num;

  public SomeClass(int n)
  {
    num = n;
  }

  public void increment(int more)
  {
    num = num + more;
  }

  public int getNum()
  {
    return num;
  }
}
```

The following code segment appears in another class.

```
SomeClass one = new SomeClass(100);
SomeClass two = new SomeClass(100);
SomeClass three = one;

one.increment(200);

System.out.println(one.getNum() + " " + two.getNum() + " " +
                   three.getNum());
```

FINAL Unit2 MCQ - Topic order

What is printed as a result of executing the code segment?

(A) `100 100 100`

(B) `300 100 100`

(C) `300 100 300` ✓

(D) `300 300 300`

154. In the following code segment, str1 and str2 are String objects. The code segment is intended to print true if str1 and str2 are non-null strings that contain the same sequence of characters. The code segment is intended to print false otherwise.

```
boolean result = false;          // line 1
if(str1 != null && str2 != null)  // line 2
{
  result = str1 == str2;         // line 4
}
System.out.println(result);
```

Which of the following best explains the error, if any, in the code segment?

(A) In line 1, result should be initialized to true.

(B) In line 2, str1 and str2 should be compared to null with the equals method.

(C) In line 4, str1 and str2 should be compared to each other with the equals method. ✓

(D) The code segment contains no errors and works as intended.

Answer C

Correct. In line 2, the String objects are correctly compared with null using the != operator. Using == in line 4 checks whether str1 and str2 refer to the same String object. To check whether they have the same value, they should be compared using the equals method.

FINAL Unit2 MCQ - Topic order

155. Consider the following code segment.

```
String str = "lookout";
String target = "o";
int j = str.indexOf(target);
while (j >= 0)
{
  str = str.substring(j);
  System.out.print(str + " ");

  str = str.substring(1);
  j = str.indexOf(target);
}
```

What is printed as a result of executing this code segment?

(A) ✓

ookout okout out

(B)

ookout out

(C)

okout out

(D)

kout ut

Answer A

Correct. The code segment repeatedly sets j to the index of the next occurrence of the target "o" in the string str. Each time an occurrence of "o" is found, the code segment prints the substring starting with that "o", then updates str to be the substring starting immediately after the "o". This is repeated until there are no "o"s remaining in str.

FINAL Unit2 MCQ - Topic order

156. Consider the following code segment.

```
String str = "abcdef";
for (int rep = 0; rep < str.length() - 1; rep++)
{
  System.out.print(str.substring(rep, rep + 2));
}
```

What is printed as a result of executing this code segment?

(A) `aabbccddeeff`

(B) `abbccddeef` ✓

(C) `abcbcdcdedef`

(D) Nothing is printed because an `IndexOutOfBoundsException` is thrown.

FINAL Unit2 MCQ - Topic order

157. Consider the following method.

```
/** Precondition: Strings one and two have the same length. */
public static String combine(String one, String two)
{
 String res = "";
 for (int k = 0; k < one.length(); k++)
 {
   if (one.substring(k, k + 1).equals(two.substring(k, k + 1)))
   {
     res += one.substring(k, k + 1);
   }
   else
   {
     res += "0";
   }
 }
 return res;
}
```

What is returned as a result of the call `combine("10110", "01100")` ?

(A) `"00000"`

(B) `"00100"` ✓

(C) `"00101"`

(D) `"10110"`

Answer B

Correct. The `combine` method compares corresponding substrings of length 1 from input strings `one` and `two`. If the substrings are the same, the substring is appended to `res`; otherwise, `"0"` is appended to `res`. The first and second characters of `res` are `"0"` because the characters in position 0 and the characters in position 1 of `one` and `two` differ. The third character of `res` is `"1"` because the characters in position 2 of `one` and `two` are both `"1"`. The fourth character in `res` is `"0"` because the characters in position 3 of `one` and `two` differ. The fifth character in `res` is `"0"` because the last characters of `one` and `two` are both `"0"`. The value `"00100"` is returned.

FINAL Unit2 MCQ - Topic order

158. Consider the following method, which is intended to count the number of times the letter `"A"` appears in the string `str`.

```
public static int countA(String str)
{
 int count = 0;

 while (str.length() > 0)
 {
   int pos = str.indexOf("A");
   if (pos >= 0)
   {
     count++;
     /* missing code */
   }
   else
   {
     return count;
   }
 }
 return count;
}
```

Which of the following should be used to replace `/* missing code */` so that method `countA` will work as intended?

(A) `str = str.substring(0, pos + 1);`

(B) `str = str.substring(pos - 1);`

(C) `str = str.substring(pos);`

(D) `str = str.substring(pos + 1);` ✓

Answer D

Correct. In the `while` loop, the variable `pos` is assigned the position of the first occurrence of `"A"` in `str`. If `pos` is less than zero, there are no more occurrences of `"A"` and the value of `count` is returned. Otherwise, the value of `count` is incremented and `str` is updated by removing the initial characters of `str`, up to and including the occurrence of `"A"` that was last included in `count`. The statement `str = str.substring(pos + 1)` accomplishes the update by selecting characters of `str` starting one character after `pos`.

FINAL Unit2 MCQ - Topic order

159. Consider the following code segment.

```
String str = "abbcdddeff";
int j = 0;
int count = 0;
while (j < str.length() - 1)
{
  if (str.substring(j, j + 1).equals(str.substring(j + 1, j + 2)))
  {
    count++;
  }
  j++;
}
System.out.println(count);
```

What is printed as a result of executing this code segment?

(A)

2

(B)

3

(C) ✓

4

(D)

6

Answer C

Correct. The code segment uses the expression str.substring(j, j + 1).equals(str.substring(j + 1, j + 2)) to compare each character in the string str to the character immediately after it. If they are equal, a counter is incremented. There are 4 places in the string where a character is equal in value to the character immediately after it (one pair of "b"s, two pairs of "d"s, and one pair of "f"s).

FINAL Unit2 MCQ - Topic order

160. Consider the following method.

```
public static int what(String str, String check)
{
 int num = -1;
 int len = check.length();
 for (int k = 0; k + len <= str.length(); k++)
 {
   String a = str.substring(k, k + len);
   if (a.equals(check))
   {
     num = k;
   }
 }
 return num;
}
```

Assume that `check` occurs at least once in `str`. Which of the following best describes the value returned by the `what` method?

(A) The number of times the string `check` occurs in `str`

(B) The index of the first occurrence of `check` inside `str`

(C) The index of the last occurrence of `check` inside `str` ✓

(D) The number of substrings in `str` with the same length as `check`

Answer C

Correct. The `for` loop assigns values to `k` from `0` to the last possible position in `str` at which `check` could appear, inclusive. In the body of the loop, the `String a` is assigned a substring of `check` that starts at position `k` and has the same length as `check`. If the substring `a` and `check` are the same, `num` is assigned `k`. When the `for` loop ends, `num` has the last value of `k` at which a match between the substring `a` and `check` was found or the index of the last occurrence of `check` in `str`.

FINAL Unit2 MCQ - Topic order

161. Consider the following method.

```
String word = "Computer Science";
String str = "";

for (int k = 0; k < word.length(); k++)
{
  if (k % 3 == 0)
  {
    str = word.substring(k, k + 1) + str;
  }
}
System.out.println(str);
```

What is printed as a result of executing the code segment?

(A)

ci tm

(B)

eeStm

(C)

ncepC

(D) ✓

eeSepC

Answer D

Correct. The expression `k % 3 == 0` is true for multiples of 3, so the single-character substrings at positions 0, 3, 6, 9, 12, and 15 are concatenated in reverse order and assigned to the variable `str`.